MÔ HÌNH PERCEPTRON

1. Từ hồi quy tuyến tính đến bài toán phân loại

Ở bài trước, ta đã xây dựng mô hình hồi quy tuyến tính để **dự đoán một đại lượng số**, chẳng hạn giá nhà hay nhiệt độ. Mô hình đó được huấn luyện theo một khung tư duy rất quan trọng:

$$\text{Mô hình} + \text{Hàm mất mát} + \text{Thuật toán tối ưu} = \text{Huấn luyện}$$

Tuy nhiên, nhiều bài toán trong thực tế không đòi hỏi một con số, mà đòi hỏi một **quyết định**: email này có phải thư rác không? Khối u này lành tính hay ác tính? Sinh viên này sẽ đậu hay rớt? Đầu ra của các bài toán này là một **nhãn** thuộc một tập hữu hạn, không phải một số thực bất kỳ. Đây là **bài toán phân loại (classification)**.

Mục tiêu của bài này là xây dựng **Perceptron**, mô hình phân loại tuyến tính đơn giản nhất, bằng cách tận dụng những gì đã học ở hồi quy tuyến tính. Trong mục 1, ta trả lời ba câu hỏi:

- Bài toán phân loại nhị phân được mô tả toán học như thế nào?
- Vì sao không thể dùng nguyên hồi quy tuyến tính để phân loại?
- Ý tưởng hình học nào cho phép ta phân loại bằng một mô hình tuyến tính?

Quy ước ký hiệu

Để thống nhất với toàn bộ bài giảng, ta quy ước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng được viết dưới dạng **vector hàng**, nhờ đó các vector đặc trưng của n mẫu có thể xếp chồng thành các hàng của ma trận đặc trưng.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng	n, d	vô hướng	n quan sát, mỗi quan sát có d đặc trưng
Vector đặc trưng (hàng)	$\mathbf{x}_i$	$1 \times (d+1)$	Đặc trưng của mẫu thứ i, phần tử đầu tiên bằng 1
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$	Hàng thứ i của $\mathbf{X}$ là $\mathbf{x}_i$
Vector tham số (cột)	$\mathbf{w}$	$(d+1) \times 1$	Gồm hệ số chệch w_0 và các trọng số $w_1, \dots, w_d$
Vector nhãn thực (cột)	$\mathbf{y}$	$n \times 1$	Nhãn thực tế $y_i \in \{-1, +1\}$ của từng mẫu
Điểm số	s_i, $\mathbf{s}$	vô hướng, $n \times 1$	$s_i = \mathbf{x}_i\mathbf{w}$ và $\mathbf{s} = \mathbf{X}\mathbf{w}$
Nhãn dự đoán	$\hat{y}_i$, $\hat{\mathbf{y}}$	vô hướng, $n \times 1$	Nhãn mà mô hình dự đoán, $\hat{y}_i \in \{-1, +1\}$

Cụ thể, với mẫu thứ i ta có vector hàng và ma trận đặc trưng:

$$\mathbf{x}_i = [1 \quad x_{i1} \quad x_{i2} \quad \cdots \quad x_{id}], \qquad \mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix}$$

Vector tham số vẫn là vector cột như bài trước:

$$\mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T$$

Lưu ý so với bài hồi quy tuyến tính. Bài trước viết vector đặc trưng là vector cột nên điểm số là $\mathbf{w}^T\mathbf{x}$. Bài này viết vector đặc trưng là vector hàng nên điểm số là $\mathbf{xw}$. Hai cách viết cho cùng một giá trị, vì $\mathbf{xw} = \mathbf{w}^T\mathbf{x}^T$. Với toàn bộ dữ liệu, công thức $\mathbf{Xw}$ không thay đổi.

1.1. Bài toán phân loại nhị phân và quy ước nhãn

Hồi quy và phân loại khác nhau ở đâu?

Hai loại bài toán này khác nhau ở **loại đầu ra** mà ta cần dự đoán:

- **Hồi quy** trả lời câu hỏi *"bao nhiêu?"*: giá một căn nhà, nhiệt độ ngày mai, điểm thi.
- **Phân loại** trả lời câu hỏi *"thuộc nhóm nào?"*: thư rác hay thư thường, đậu hay rớt, bệnh hay không bệnh.

Khi chỉ có **hai nhóm** để lựa chọn, ta có **bài toán phân loại nhị phân (binary classification)**. Hai nhóm này thường được gọi là:

- **lớp dương (positive class)**, ký hiệu nhãn $y = +1$;
- **lớp âm (negative class)**, ký hiệu nhãn $y = -1$.

Cần lưu ý rằng "dương" và "âm" chỉ là tên gọi. Chúng không có nghĩa là "tốt" hay "xấu". Thông thường, lớp dương là lớp mà ta quan tâm cần phát hiện, ví dụ thư rác hay bệnh nhân mắc bệnh.

Mô tả bài toán

Giả sử ta có bộ dữ liệu huấn luyện gồm n quan sát:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}, \qquad y_i \in \{-1, +1\}$$

Trong đó $\mathbf{x}_i$ là vector đặc trưng (hàng) của quan sát thứ i và y_i là nhãn thực tế của nó. Khác với hồi quy, y_i **chỉ nhận một trong hai giá trị** -1 hoặc $+1$.

Bài toán phân loại nhị phân là: **tìm một quy tắc dự đoán nhãn** $\hat{y} \in \{-1, +1\}$ **từ đặc trưng x sao cho nhãn dự đoán trùng với nhãn thực tế trên càng nhiều mẫu càng tốt.**

Ví dụ xuyên suốt: dự đoán sinh viên đậu hay rớt

Ta xét một ví dụ nhỏ sẽ được dùng lại trong suốt bài giảng. Mỗi sinh viên được mô tả bởi hai đặc trưng: x_1 là số giờ tự học mỗi tuần và x_2 là số buổi vắng mặt. Nhãn $y = +1$ nếu đậu và $y = -1$ nếu rớt.

Sinh viên	x_1 (giờ tự học)	x_2 (buổi vắng)	y (kết quả)
1	2	4	-1 (rớt)
2	3	3	-1 (rớt)
3	4	5	-1 (rớt)
4	6	1	$+1$ (đậu)
5	8	2	$+1$ (đậu)
6	9	0	$+1$ (đậu)

Ở đây $n = 6$ và $d = 2$. Thêm cột 1 vào đặc trưng, ta tổ chức dữ liệu thành ma trận đặc trưng và vector nhãn:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 1 \\ 1 & 8 & 2 \\ 1 & 9 & 0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}$$

Mỗi hàng của $\mathbf{X}$ là vector đặc trưng của một sinh viên, ví dụ $\mathbf{x}_4 = [1 \quad 6 \quad 1]$.

Quy ước nhãn

Trong tài liệu về phân loại, nhãn nhị phân đôi khi được mã hóa là $\{0,1\}$, đôi khi là $\{-1,+1\}$. Đây chỉ là hai cách mã hóa khác nhau cho cùng một ý nghĩa, và chuyển qua lại bằng phép biến đổi $y' = 2y - 1$ (từ $\{0,1\}$ sang $\{-1,+1\}$). Trong bài giảng này, ta thống nhất dùng

$$y_i \in \{-1,+1\}, \qquad \hat{y}_i \in \{-1,+1\}$$

Lý do chọn $\{-1,+1\}$ là vì cách mã hóa này rất thuận lợi cho Perceptron:

- Nhãn dự đoán có thể lấy trực tiếp từ **dấu** của điểm số, bằng **hàm dấu (sign)**, như ta sẽ thấy ở mục 2.
- Việc dự đoán đúng hay sai được gói gọn trong **một phép nhân**: tích $y_i\hat{y}_i$ bằng $+1$ nếu đúng và -1 nếu sai.

y_i	$\hat{y}_i$	$y_i\hat{y}_i$	Ý nghĩa
$+1$	$+1$	$+1$	Dự đoán đúng
-1	-1	$+1$	Dự đoán đúng
$+1$	-1	-1	Sai: bỏ sót mẫu thuộc lớp dương
-1	$+1$	-1	Sai: báo nhầm mẫu thuộc lớp âm thành lớp dương

Ở các mục sau, ta sẽ thấy tích giữa nhãn thực và **điểm số** của mô hình, tức $y_i s_i$, vừa cho biết mô hình đúng hay sai, vừa cho biết mô hình "đúng chắc chắn" đến mức nào. Đây là lợi thế lớn khi xây dựng hàm mất mát và quy tắc cập nhật tham số của Perceptron.

1.2. Vì sao không dùng thẳng hồi quy tuyến tính?

Một ý tưởng tự nhiên

Một ý tưởng rất tự nhiên là: ta đã có công cụ hồi quy tuyến tính rất mạnh, vậy hãy **coi nhãn -1 và $+1$ như hai con số** và dùng hồi quy tuyến tính để dự đoán chúng. Sau khi huấn luyện, mô hình cho ra điểm số

$$s = \mathbf{x}\mathbf{w} = w_0 + w_1 x_1 + \cdots + w_d x_d$$

Vì nhãn chỉ là -1 hoặc $+1$, ta chọn ngưỡng ở giữa hai giá trị này, tức là 0:

$$\hat{y} = \begin{cases} +1 & \text{nếu } s \geq 0 \\ -1 & \text{nếu } s < 0 \end{cases}$$

Cách làm này nghe có vẻ hợp lý. Ta hãy thử trên một ví dụ nhỏ để xem nó có thực sự hoạt động tốt không.

Ví dụ: chỉ có một đặc trưng

Xét 8 sinh viên, đặc trưng duy nhất là số giờ tự học x mỗi tuần. Bốn sinh viên học 1,2,3,4 giờ đều rớt ($y = -1$). Bốn sinh viên học 5,6,7,8 giờ đều đậu ($y = +1$). Huấn luyện hồi quy tuyến tính với hàm mất mát MSE trên dữ liệu này, ta thu được

$$s = -1{,}714 + 0{,}381\, x$$

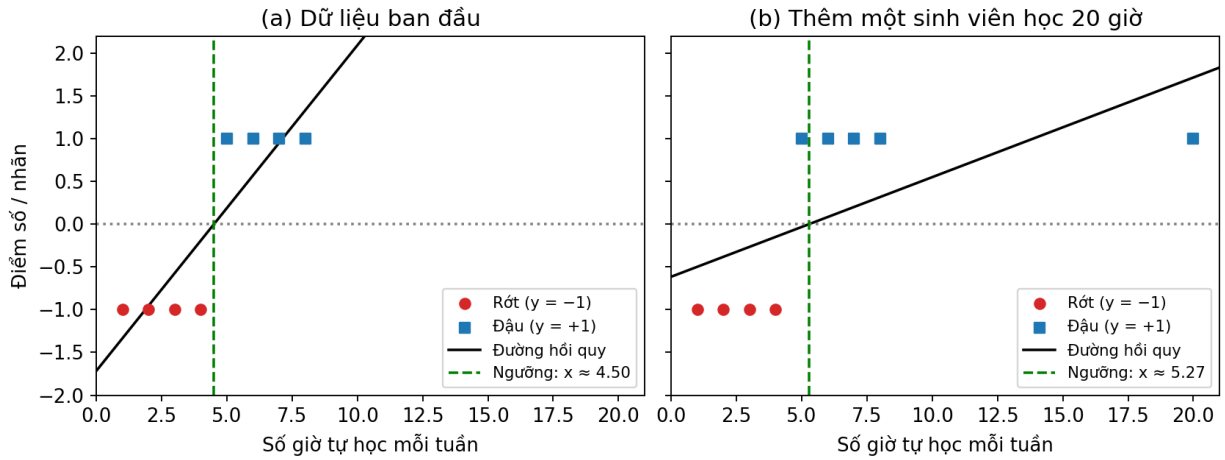
Ngưỡng $s = 0$ tương ứng với $x = 4{,}5$. Mô hình phân loại đúng cả 8 sinh viên (Hình 1a). Đến đây mọi thứ vẫn ổn. Tuy nhiên, hãy quan sát kỹ hơn hai điều.

Quan sát 1: Đầu ra không phải là nhãn. Tại $x = 1$, mô hình cho $s \approx -1{,}33$. Tại $x = 8$, mô hình cho $s \approx 1{,}33$. Những giá trị này nằm ngoài đoạn $[-1,1]$ nên không có ý nghĩa như một nhãn. Mô hình hồi quy chỉ cho ra một điểm số liên tục. Nhãn chỉ xuất hiện nhờ bước đặt ngưỡng mà ta gắn thêm vào một cách thủ công.

Quan sát 2: Một mẫu "rất đúng" làm hỏng mô hình. Giả sử có thêm một sinh viên học 20 giờ mỗi tuần và dĩ nhiên đậu ($y = +1$). Mẫu này hoàn toàn phù hợp với xu hướng chung, ta mong nó không gây rắc rối gì. Nhưng khi huấn luyện lại, mô hình trở thành

$$s = -0{,}614 + 0{,}117\, x$$

Ngưỡng $s = 0$ bây giờ ứng với $x \approx 5{,}27$ (Hình 1b). Sinh viên học 5 giờ (thực tế đậu) có $s \approx -0{,}03 < 0$ nên bị dự đoán thành **rớt**. Mô hình đã phân loại sai một mẫu mà trước đó nó phân loại đúng, chỉ vì ta thêm một mẫu dễ.



Hình 1. Hồi quy tuyến tính dùng để phân loại: (a) trên dữ liệu ban đầu, ngưỡng tại $x = 4{,}5$; (b) sau khi thêm sinh viên học 20 giờ, đường hồi quy bị kéo thấp xuống và ngưỡng dịch sang $x \approx 5{,}27$.

Nguyên nhân: hàm mất mát không đo đúng điều ta quan tâm

Hiện tượng trên xảy ra vì hồi quy tuyến tính tối thiểu hóa MSE, tức là cố gắng đưa điểm số s **càng gần -1 hoặc $+1$ càng tốt**, chứ không phải đưa nó về **đúng phía** của ngưỡng.

Với sinh viên học 20 giờ, mô hình mới cho $s \approx 1{,}72$. Điểm số này đã nằm ở phía "đậu" của ngưỡng 0 nên phân loại hoàn toàn đúng. Tuy vậy, MSE vẫn **phạt** mẫu này vì $1{,}72 \neq 1$. Để giảm bớt hình phạt đó, mô hình đánh đổi bằng cách hạ thấp độ dốc của đường thẳng, kéo theo các mẫu khác bị lệch ngưỡng.

Nói cách khác, cái ta thực sự quan tâm là **số mẫu bị phân loại đúng**, trong khi MSE đo **khoảng cách tới các giá trị -1 và $+1$**. Hai điều này không đồng nhất: giảm MSE chưa chắc làm tăng số mẫu được phân loại đúng.

Ngoài ra, nếu bài toán có nhiều hơn hai lớp, việc gán nhãn bằng các số 1,2,3, ... còn vô tình áp đặt một **thứ tự** giữa các lớp, điều không có trong bản chất của bài toán.

Kết luận

Cần phân biệt rõ: hồi quy tuyến tính **không sai**, nó chỉ **không được thiết kế** cho mục tiêu phân loại. Đặt trong khung "Mô hình, Hàm mất mát, Thuật toán tối ưu", ta thấy hai thành phần cần thay đổi:

	Hồi quy tuyến tính	Bài toán phân loại cần
Đầu ra của mô hình	Một số thực s	Một nhãn $\hat{y} \in \{-1, +1\}$
Câu hỏi của bài toán	Bao nhiêu?	Thuộc nhóm nào?
Tiêu chí đánh giá	Khoảng cách $\lvert y - s \rvert$ nhỏ	Dự đoán đúng nhãn càng nhiều càng tốt

Vì vậy, ta cần:

1. một mô hình có đầu ra là **nhãn** ngay từ đầu, thay vì chỉ cho điểm số rồi đặt ngưỡng thủ công;
2. một tiêu chí huấn luyện phản ánh đúng việc **đúng hay sai**, thay vì khoảng cách tới -1 và $+1$.

Điểm tích cực là ta không cần bỏ đi mọi thứ đã học. Biểu thức tuyến tính $s = \mathbf{xw}$ vẫn rất hữu ích, và mục tiếp theo cho thấy nó có một ý nghĩa hình học rõ ràng khi dùng để phân loại.

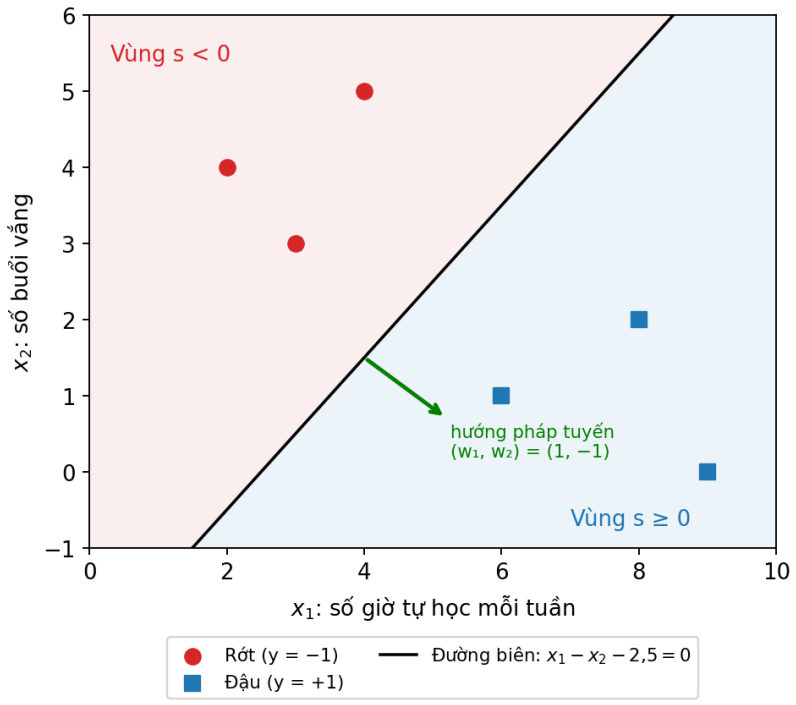
1.3. Ý tưởng phân loại bằng một đường thẳng hoặc siêu phẳng

Nhìn dữ liệu dưới dạng các điểm trong không gian

Mỗi mẫu có d đặc trưng có thể được xem là một **điểm** trong không gian d chiều. Trong ví dụ sinh viên, mỗi sinh viên là một điểm trên mặt phẳng với hai tọa độ (x_1, x_2). Hình 2 biểu diễn 6 sinh viên trên mặt phẳng này.

Quan sát hình, ta thấy các sinh viên rớt (chấm tròn đỏ) nằm ở phía trên bên trái, còn các sinh viên đậu (ô vuông xanh) nằm ở phía dưới bên phải. Hai nhóm này có thể được **ngăn cách bằng một đường thẳng**. Đây chính là ý tưởng cốt lõi của phân loại tuyến tính:

> **Tìm một đường thẳng (hoặc tổng quát hơn là một siêu phẳng) chia không gian đặc trưng thành hai phía, mỗi phía ứng với một lớp.**



Hình 2. Dữ liệu sinh viên đậu/rớt trên mặt phẳng đặc trưng và một đường thẳng phân chia hai lớp. Mũi tên xanh là hướng pháp tuyến của đường thẳng, chỉ về phía tăng của điểm số s.

Phương trình của đường biên

Trong mặt phẳng hai chiều, một đường thẳng có phương trình

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

Với vector đặc trưng hàng $\mathbf{x} = [1 \quad x_1 \quad x_2]$ và vector tham số $\mathbf{w} = [w_0 \quad w_1 \quad w_2]^T$, phương trình này được viết gọn là

$$\mathbf{xw} = 0$$

Tổng quát, với d đặc trưng, tập hợp các điểm thỏa mãn phương trình

$$w_0 + w_1 x_1 + \cdots + w_d x_d = 0 \quad \Leftrightarrow \quad \mathbf{xw} = 0$$

được gọi là **siêu phẳng (hyperplane)**. Khi $d = 1$ nó là một điểm, khi $d = 2$ là một đường thẳng, khi $d = 3$ là một mặt phẳng, và khi $d > 3$ ta không thể vẽ ra nhưng vẫn gọi là siêu phẳng. Siêu phẳng này được gọi là **đường biên quyết định (decision boundary)** vì nó là ranh giới giữa hai quyết định.

Ta thấy biểu thức **xw** chính là biểu thức quen thuộc của hồi quy tuyến tính. Điểm khác biệt nằm ở **cách sử dụng** nó: thay vì coi **xw** là giá trị dự đoán, ta dùng nó để cho biết điểm **x** nằm ở phía nào của đường biên.

Điểm số cho biết điểm nằm ở phía nào

Với mỗi mẫu **x**, ta tính **điểm số (score)**

$$s = \mathbf{xw}$$

Có ba khả năng:

- $s = 0$: điểm nằm **trên** đường biên;
- $s > 0$: điểm nằm ở **một phía** của đường biên;
- $s < 0$: điểm nằm ở **phía còn lại**.

Từ đó, ta có quy tắc gán nhãn dựa vào dấu của điểm số:

$$\hat{y} = \begin{cases} +1 & \text{nếu } s \geq 0 \\ -1 & \text{nếu } s < 0 \end{cases}$$

Đây mới chỉ là ý tưởng. Mục 2 sẽ hình thức hóa nó thành mô hình Perceptron với hàm dấu sign và chốt cách xử lý trường hợp $s = 0$.

Điểm khác biệt quan trọng so với hồi quy: **ngưỡng ở đây là 0 và được tích hợp sẵn trong mô hình**. Hệ số chệch w_0 đảm nhiệm việc dịch chuyển đường biên, nên ta không cần chọn một ngưỡng riêng.

Ví dụ số với dữ liệu sinh viên

Chọn vector tham số

$$\mathbf{w} = \begin{bmatrix} -2{,}5 \\ 1 \\ -1 \end{bmatrix}$$

Khi đó đường biên có phương trình $-2{,}5 + x_1 - x_2 = 0$, hay $x_2 = x_1 - 2{,}5$ (đường đen trong Hình 2). Điểm số của cả 6 sinh viên được tính đồng thời bằng một phép nhân ma trận:

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 1 \\ 1 & 8 & 2 \\ 1 & 9 & 0 \end{bmatrix} \begin{bmatrix} -2{,}5 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -4{,}5 \\ -2{,}5 \\ -3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix}$$

Áp dụng quy tắc dựa vào dấu của điểm số, ta thu được bảng kết quả. Cột cuối là tích $y_i s_i$ giữa nhãn thực và điểm số:

Sinh viên	s	$\hat{y}$	y	$y \cdot s$	Đúng/Sai
1	$-4{,}5$	-1	-1	4,5	Đúng
2	$-2{,}5$	-1	-1	2,5	Đúng
3	$-3{,}5$	-1	-1	3,5	Đúng
4	2,5	$+1$	$+1$	2,5	Đúng
5	3,5	$+1$	$+1$	3,5	Đúng
6	6,5	$+1$	$+1$	6,5	Đúng

Với bộ tham số này, mô hình phân loại đúng cả 6 sinh viên, và ta không cần đến bất kỳ ngưỡng thủ công nào. Hãy chú ý cột $y \cdot s$: nó **dương ở mọi mẫu được phân loại đúng**. Thật vậy, nếu $y = +1$ thì đúng khi $s > 0$, còn nếu $y = -1$ thì đúng khi $s < 0$, và cả hai trường hợp đều cho $y \cdot s > 0$. Ngược lại, mẫu bị phân loại sai sẽ có $y \cdot s < 0$. Như vậy, **dấu của $y \cdot s$ cho biết đúng hay sai**, còn **độ lớn của nó cho biết mô hình đúng (hoặc sai) nhiều đến mức nào**.

Ý nghĩa hình học của các tham số

Tách vector tham số thành hai phần: hệ số chệch w_0 và các trọng số $w_1, \dots, w_d$. Đặt $\tilde{\mathbf{w}} = [w_1 \quad \cdots \quad w_d]^T$. Khi đó:

- **$\tilde{\mathbf{w}}$ là vector pháp tuyến** của đường biên, tức là vuông góc với đường biên và chỉ về phía điểm số s **tăng**. Trong ví dụ, $\tilde{\mathbf{w}} = [1 \quad -1]^T$, mũi tên xanh ở Hình 2 chỉ về phía các sinh viên đậu.
- w_0 **quyết định vị trí** của đường biên: thay đổi w_0 làm đường biên tịnh tiến song song với chính nó, còn thay đổi $\tilde{\mathbf{w}}$ làm đường biên xoay.

Khoảng cách từ điểm $\mathbf{x}$ đến đường biên là

$$\text{khoảng cách} = \frac{|s|}{\| \tilde{\mathbf{w}} \|} = \frac{|\mathbf{x}\mathbf{w}|}{\sqrt{w_1^2 + \cdots + w_d^2}}$$

Ví dụ, sinh viên 4 có $s = 2{,}5$ và $\| \tilde{\mathbf{w}} \| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$, nên

$$\text{khoảng cách} = \frac{2{,}5}{\sqrt{2}} \approx 1{,}77$$

Nếu thêm nhãn vào, đại lượng

$$\frac{y_i s_i}{\| \tilde{\mathbf{w}} \|}$$

là **khoảng cách có dấu**: dương nếu mẫu nằm đúng phía của đường biên và âm nếu nằm sai phía. Điểm càng xa đường biên (về phía đúng) thì ta càng "tự tin" về nhãn dự đoán. Ngược lại, điểm sát đường biên chỉ cần dịch chuyển một chút là đổi nhãn.

Khi nào có thể phân loại hoàn hảo bằng một đường thẳng?

Không phải bộ dữ liệu nào cũng có thể chia bằng một đường thẳng. Ta gọi dữ liệu là **phân tách tuyến tính được (linearly separable)** nếu tồn tại một vector tham số $\mathbf{w}$ sao cho mọi mẫu đều nằm đúng phía của đường biên. Nhờ nhãn $\{-1, +1\}$, điều kiện này được viết chỉ bằng **một bất đẳng thức**:

$$y_i \, (\mathbf{x}_i \mathbf{w}) > 0 \qquad \text{với mọi } i = 1, \dots, n$$

Ở đây ta dùng bất đẳng thức chặt để mẫu nằm đúng trên đường biên ($s = 0$) không được coi là nằm đúng phía một cách chắc chắn. Dữ liệu sinh viên ở trên thỏa mãn điều kiện này, vì cột $y \cdot s$ trong bảng đều dương. Có những bộ dữ liệu đơn giản không thỏa mãn, tiêu biểu là bài toán XOR mà ta sẽ gặp ở phần sau, và đó chính là giới hạn cơ bản của mọi mô hình phân loại tuyến tính.

Có rất nhiều đường thẳng đúng: làm sao chọn một đường?

Đường biên ở trên không phải là đường duy nhất phân loại đúng. Thử với

$$\mathbf{w}' = \begin{bmatrix} -2 \\ 1 \\ -0{,}8 \end{bmatrix}$$

Ta tính được $\mathbf{X}\mathbf{w}' = [-3{,}2 \quad -1{,}4 \quad -2 \quad 3{,}2 \quad 4{,}4 \quad 7]^T$. Nhân từng phần tử với nhãn tương ứng, ta được các giá trị $y_i s_i$ lần lượt là $3{,}2;\ 1{,}4;\ 2;\ 3{,}2;\ 4{,}4;\ 7$, tất cả đều dương, nên cả 6 sinh viên đều được phân loại đúng. Như vậy, có vô số bộ tham số $\mathbf{w}$ cùng phân loại đúng dữ liệu này.

Tình huống này gợi lại câu hỏi đã gặp ở bài hồi quy tuyến tính:

Các tham số w lấy ở đâu?

Ta cần một **quá trình huấn luyện** để máy tính tự tìm ra $\mathbf{w}$ từ dữ liệu, bắt đầu từ một giá trị khởi tạo rồi cải thiện dần, tương tự cách Gradient Descent cải thiện tham số ở bài trước. Mô hình thực hiện ý tưởng này chính là **Perceptron**, đối tượng của mục 2 và mục 3.

Tóm tắt mục 1

- **Phân loại nhị phân** dự đoán một nhãn $y \in \{-1, +1\}$ thay vì một số thực. Với cách mã hóa này, tích $y_i \hat{y}_i = +1$ khi đúng và -1 khi sai.
- Dùng thẳng **hồi quy tuyến tính rồi đặt ngưỡng** là khả thi nhưng không phù hợp. Đầu ra không phải nhãn, và hàm mất mát MSE đo khoảng cách tới -1 và $+1$ thay vì đo việc đúng hay sai, nên một mẫu dễ cũng có thể làm lệch ngưỡng.
- Ý tưởng phân loại tuyến tính: tìm **siêu phẳng** $\mathbf{xw} = 0$ chia không gian đặc trưng thành hai phía. **Dấu của điểm số** $s = \mathbf{xw}$ cho biết nhãn, còn **dấu của** $y \cdot s$ cho biết mô hình đúng hay sai.
- Vector $\tilde{\mathbf{w}}$ là **pháp tuyến** của đường biên, còn w_0 quyết định **vị trí**.
- Dữ liệu **phân tách tuyến tính được** khi tồn tại $\mathbf{w}$ thỏa $y_i \, \mathbf{x}_i \mathbf{w} > 0$ với mọi i. Khi đó có vô số $\mathbf{w}$ như vậy, và ta cần một thuật toán huấn luyện để tìm ra một $\mathbf{w}$.

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ và một sinh viên có vector đặc trưng $\mathbf{x} = [1 \quad 5 \quad 2]$, hãy tính điểm số s và nhãn dự đoán $\hat{y}$.
2. Một mẫu có nhãn thực $y = -1$ nhưng bị mô hình dự đoán $\hat{y} = +1$. Tích $y\hat{y}$ bằng bao nhiêu? Đây là loại sai nào?
3. Với $\mathbf{w}$ như ở câu 1 và sinh viên 2 trong bảng dữ liệu ($\mathbf{x}_2 = [1 \quad 3 \quad 3]$, $y_2 = -1$), hãy tính s và $y \cdot s$. Mô hình đúng hay sai trên mẫu này?
4. Giải thích bằng lời: vì sao việc thêm một mẫu "hiển nhiên đúng" (sinh viên học 20 giờ và đậu) lại làm hồi quy tuyến tính phân loại sai một mẫu khác?
5. Đường thẳng $2x_1 - 2x_2 - 5 = 0$ có cùng đường biên với $x_1 - x_2 - 2{,}5 = 0$ không? Vector tham số tương ứng $[-5 \quad 2 \quad -2]^T$ có cho cùng nhãn dự đoán trên 6 sinh viên không?

Gợi ý đáp án

1. $s = -2{,}5 + 1 \cdot 5 + (-1) \cdot 2 = 0{,}5 \geq 0$ nên $\hat{y} = +1$ (dự đoán đậu).
2. $y\hat{y} = (-1)(+1) = -1$. Đây là lỗi **báo nhầm**: mẫu thuộc lớp âm nhưng bị dự đoán thành lớp dương.
3. $s = -2{,}5 + 3 - 3 = -2{,}5$ và $y \cdot s = (-1)(-2{,}5) = 2{,}5 > 0$. Mô hình **đúng** trên mẫu này, vì $s < 0$ ứng với $\hat{y} = -1 = y$.
4. Mẫu mới có điểm số hồi quy đã ở đúng phía ngưỡng nhưng MSE vẫn phạt vì điểm số khác 1. Để giảm phạt, đường hồi quy bị kéo thấp xuống và ngưỡng dịch sang phải, làm sinh viên học 5 giờ rơi xuống phía "rớt".
5. Có. Phương trình thứ hai là phương trình thứ nhất nhân với 2 nên hai đường trùng nhau. Điểm số mới bằng đúng 2 lần điểm số cũ, cùng dấu, nên nhãn dự đoán không đổi. Nhận xét này cho thấy $\mathbf{w}$ và $c\,\mathbf{w}$ với $c > 0$ định nghĩa cùng một bộ phân loại.

2. Mô hình Perceptron

Ở mục 1, ta đã thấy ý tưởng phân loại bằng một siêu phẳng: tính điểm số $s = \mathbf{xw}$ rồi dựa vào **dấu** của s để chọn nhãn. Mục này biến ý tưởng đó thành một mô hình hoàn chỉnh, gọi là **Perceptron**.

Perceptron do Frank Rosenblatt đề xuất năm 1958 và được xem là **nơ-ron nhân tạo** đầu tiên có thể học từ dữ liệu. Mô hình rất đơn giản, nhưng nó là nền tảng của Logistic Regression và các mạng nơ-ron hiện đại. Trong mục này, ta trả lời ba câu hỏi:

- Perceptron nhận đầu vào, xử lý và đưa ra quyết định như thế nào?
- Mô hình được viết gọn bằng vector và ma trận ra sao?
- Perceptron chia không gian đặc trưng như thế nào, và các tham số điều khiển việc chia đó ra sao?

Ta tiếp tục dùng quy ước ký hiệu ở mục 1: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng** với phần tử đầu tiên bằng 1, vector tham số $\mathbf{w}$ là **vector cột**, và nhãn $y_i, \hat{y}_i \in \{-1, +1\}$.

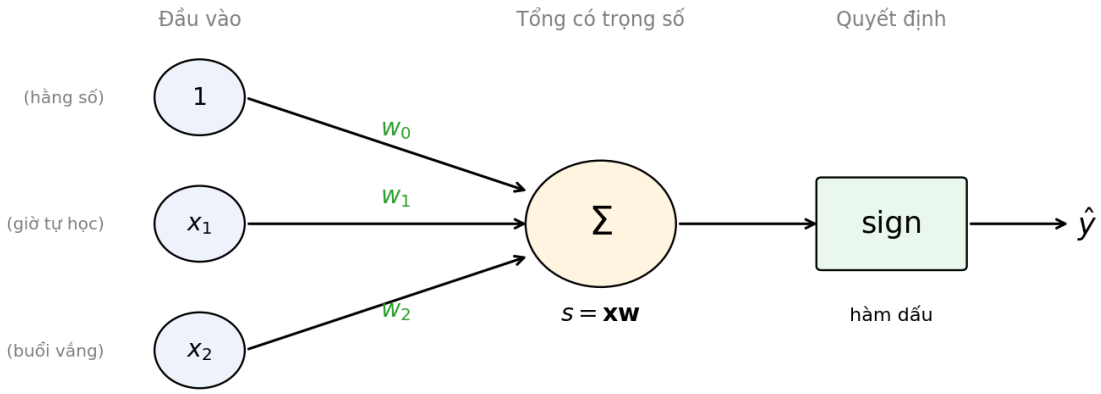
2.1. Tổng có trọng số, hệ số chệch và hàm dấu

Ý tưởng hoạt động: một "máy ra quyết định" ba bước

Có thể hình dung Perceptron như một người ra quyết định bằng cách **cân nhắc nhiều yếu tố**, mỗi yếu tố có một mức độ quan trọng riêng. Quy trình gồm ba bước:

1. **Nhận đặc trưng.** Mô hình nhận vào các đặc trưng $x_1, x_2, \ldots, x_d$ của một mẫu.
2. **Tính điểm số.** Mỗi đặc trưng được nhân với một **trọng số** thể hiện mức độ quan trọng, rồi cộng tất cả lại và cộng thêm một hằng số điều chỉnh.
3. **Ra quyết định.** Nếu điểm số không âm, mô hình trả lời "lớp dương" ($\hat{y} = +1$). Ngược lại, mô hình trả lời "lớp âm" ($\hat{y} = -1$).

Hình 3 minh họa quy trình này với ví dụ sinh viên đậu/rớt ($d = 2$).



Hình 3. Cấu trúc của Perceptron với hai đặc trưng. Các đầu vào được nhân với trọng số, cộng lại thành điểm số s, sau đó hàm dấu chuyển điểm số thành nhãn ŷ.

Hàm tổng hợp có trọng số và hệ số chệch

Với một mẫu có vector đặc trưng $\mathbf{x} = [1 \quad x_1 \quad \cdots \quad x_d]$, Perceptron tính **điểm số (score)** là tổng có trọng số của các đặc trưng, cộng với hệ số chệch:

$$s = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = w_0 + \sum_{j=1}^{d} w_j \, x_j$$

Trong đó:

- $w_1, \ldots, w_d$ là các **trọng số (weight)**. Trọng số w_j cho biết đặc trưng x_j ảnh hưởng thế nào đến điểm số: nếu $w_j > 0$ thì x_j tăng làm s tăng, nếu $w_j < 0$ thì x_j tăng làm s giảm, và nếu $w_j = 0$ thì đặc trưng x_j không có ảnh hưởng.
- w_0 là **hệ số chệch (bias)**. Nó là điểm số của mẫu khi mọi đặc trưng bằng 0, và đóng vai trò như một "mức khởi điểm" của mô hình.

Nhờ thêm số 1 vào đầu vector **x** (như đã làm ở hồi quy tuyến tính), toàn bộ biểu thức trên được viết gọn thành

$$s = \mathbf{xw}, \qquad \mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T$$

Ý nghĩa trong ví dụ sinh viên. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ ta có

$$s = -2{,}5 + 1 \cdot x_1 - 1 \cdot x_2$$

Có thể đọc công thức này như sau: mỗi giờ tự học làm điểm số tăng 1, mỗi buổi vắng làm điểm số giảm 1, và mô hình **bắt đầu từ mức** $-2{,}5$, nghĩa là sinh viên phải "tích lũy" đủ giờ học để vượt qua mức khởi điểm âm này thì mới có điểm số không âm. Hệ số chệch w_0 vì vậy chính là thứ quyết định **độ khó** của việc được xếp vào lớp dương.

Điều này giống hồi quy tuyến tính. Biểu thức $s = \mathbf{xw}$ hoàn toàn giống biểu thức của hồi quy tuyến tính. Điểm khác biệt nằm ở bước tiếp theo: Perceptron không dừng ở điểm số mà đưa nó qua một hàm kích hoạt để thu được nhãn.

Hàm kích hoạt dấu và quy tắc dự đoán

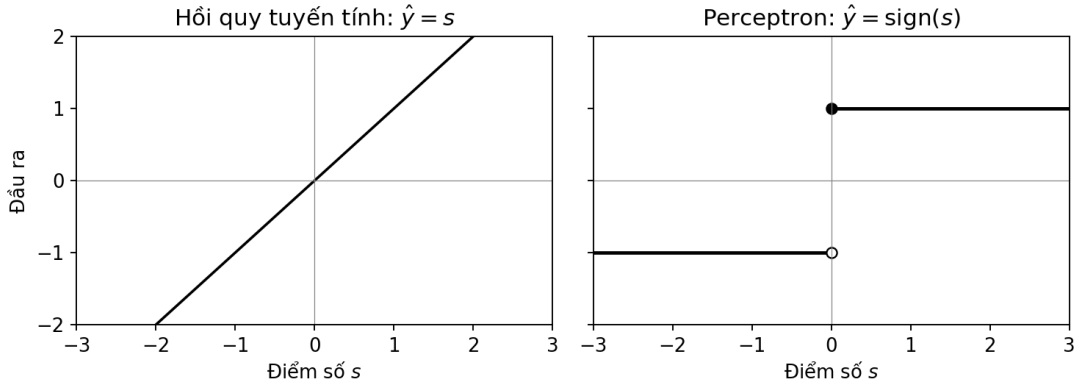
Để chuyển điểm số s (một số thực) thành nhãn $\hat{y} \in \{-1, +1\}$, Perceptron dùng **hàm dấu (sign function)** làm hàm kích hoạt:

$$\text{sign}(s) = \begin{cases} +1 & \text{nếu } s \geq 0 \\ -1 & \text{nếu } s < 0 \end{cases}$$

Khi đó **quy tắc dự đoán** của Perceptron là

$$\hat{y} = \text{sign}(s) = \text{sign}(\mathbf{xw}) = \begin{cases} +1 & \text{nếu } \mathbf{xw} \geq 0 \\ -1 & \text{nếu } \mathbf{xw} < 0 \end{cases}$$

Hình 4 so sánh đầu ra của hồi quy tuyến tính và của Perceptron theo cùng một điểm số s. Hồi quy tuyến tính giữ nguyên điểm số, còn Perceptron "bẻ" điểm số về đúng một trong hai giá trị -1 hoặc $+1$.



Hình 4. Hàm đồng nhất của hồi quy tuyến tính và hàm dấu của Perceptron. Chấm đặc nghĩa là giá trị tại $s = 0$ thuộc về đoạn $\hat{y} = +1$.

Có thể tóm gọn: **Perceptron = hồi quy tuyến tính (tính điểm số) + hàm dấu (đưa ra nhãn).**

Quy ước tại $s = 0$. Trong toán học thông thường, $\text{sign}(0) = 0$. Nhưng nhãn của ta chỉ có thể là -1 hoặc $+1$, nên ta quy ước điểm nằm đúng trên đường biên ($s = 0$) được gán nhãn $+1$, như trong công thức ở trên. Một số tài liệu chọn nhãn -1, và sự khác biệt này không quan trọng về mặt ý tưởng, nhưng ta cần chọn một quy ước và giữ nhất quán. Quy ước này có một hệ quả sẽ gặp ở mục 3: khi khởi tạo $\mathbf{w} = \mathbf{0}$, mọi mẫu đều có $s = 0$ nên đều bị dự đoán là $+1$.

Ví dụ. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$:

- Sinh viên học 5 giờ, vắng 2 buổi, $\mathbf{x} = [1 \quad 5 \quad 2]$: $s = -2{,}5 + 5 - 2 = 0{,}5 \geq 0$, nên $\hat{y} = +1$ (đậu).

- Sinh viên học 3 giờ, vắng 4 buổi, $\mathbf{x} = [1 \quad 3 \quad 4]$: $s = -2{,}5 + 3 - 4 = -3{,}5 < 0$, nên $\hat{y} = -1$ (rớt).
- Sinh viên học 4,5 giờ, vắng 2 buổi, $\mathbf{x} = [1 \quad 4{,}5 \quad 2]$: $s = -2{,}5 + 4{,}5 - 2 = 0$. Theo quy ước, $\hat{y} = +1$.

Một hệ quả quan trọng: hàm dấu không có đạo hàm hữu ích

Ở bài hồi quy tuyến tính, ta huấn luyện mô hình bằng Gradient Descent, vốn cần **đạo hàm** của hàm mất mát theo tham số. Với Perceptron, hàm dấu là hàm hằng ở cả hai phía của $s = 0$ nên đạo hàm của nó bằng 0 tại mọi điểm $s \neq 0$ (và không xác định tại $s = 0$). Nếu ta cố gắng tối thiểu hóa MSE giữa y và $\hat{y} = \text{sign}(\mathbf{xw})$, gradient sẽ bằng 0 hầu như ở mọi nơi, nên Gradient Descent **không có hướng nào để đi**.

Vì vậy, Perceptron cần một **cách huấn luyện khác**, bám sát việc "đúng hay sai" của từng mẫu, mà tích $y_i s_i$ ở mục 1 chính là công cụ phù hợp. Đây là nội dung của mục 3.

2.2. Biểu diễn vector và ma trận của mô hình

Một mẫu

Với một mẫu, mô hình chỉ gồm hai bước: nhân vector hàng $\mathbf{x}_i$ với vector cột $\mathbf{w}$, rồi đưa kết quả qua hàm dấu:

$$s_i = \mathbf{x}_i \mathbf{w}, \qquad \hat{y}_i = \text{sign}(s_i)$$

Về kích thước, $\mathbf{x}_i$ có kích thước $1 \times (d+1)$ và $\mathbf{w}$ có kích thước $(d+1) \times 1$, nên tích $\mathbf{x}_i \mathbf{w}$ là một **số** (ma trận 1×1), đúng với điểm số s_i.

Toàn bộ n mẫu cùng lúc

Vì mỗi hàng của ma trận đặc trưng $\mathbf{X}$ chính là một vector $\mathbf{x}_i$, một phép nhân ma trận cho ta điểm số của **tất cả các mẫu**:

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} \mathbf{x}_1 \mathbf{w} \\ \mathbf{x}_2 \mathbf{w} \\ \vdots \\ \mathbf{x}_n \mathbf{w} \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}$$

Sau đó ta áp dụng hàm dấu **lên từng phần tử** của $\mathbf{s}$ để được vector nhãn dự đoán:

$$\hat{\mathbf{y}} = \text{sign}(\mathbf{X}\mathbf{w}) = \begin{bmatrix} \text{sign}(s_1) \\ \text{sign}(s_2) \\ \vdots \\ \text{sign}(s_n) \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

Cách viết này có ba lợi ích: công thức ngắn gọn, thống nhất với hồi quy tuyến tính (chỉ thêm hàm dấu), và khi lập trình ta có thể tính cho cả tập dữ liệu bằng một phép nhân ma trận thay vì một vòng lặp qua từng mẫu.

Đại lượng $m_i = y_i s_i$: mẫu đúng hay sai, và đúng đến mức nào

Ở mục 1, ta đã thấy tích giữa nhãn thực và điểm số cho biết mô hình đúng hay sai. Ta đặt tên cho đại lượng này, gọi là **margin (có dấu)** của mẫu thứ i:

$$m_i = y_i\, s_i = y_i\, (\mathbf{x}_i \mathbf{w})$$

- $m_i > 0$: mô hình dự đoán **đúng** mẫu i, và m_i càng lớn thì mẫu càng nằm sâu trong vùng đúng.
- $m_i < 0$: mô hình dự đoán **sai** mẫu i, và $|m_i|$ càng lớn thì mẫu càng nằm sâu trong vùng sai.
- $m_i = 0$: mẫu nằm đúng trên đường biên. Theo quy ước $\text{sign}(0) = +1$, mẫu này được dự đoán đúng nếu $y_i = +1$ và sai nếu $y_i = -1$.

Với toàn bộ dữ liệu, ta có thể gom các margin thành một vector bằng cách nhân **từng phần tử** của $\mathbf{y}$ với phần tử tương ứng của $\mathbf{s}$ (phép nhân Hadamard, ký hiệu $\odot$):

$$\mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} y_1 s_1 \\ y_2 s_2 \\ \vdots \\ y_n s_n \end{bmatrix}$$

Bảng sau tổng hợp kích thước của các đại lượng:

Đại lượng	Biểu thức	Kích thước
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$
Vector tham số	$\mathbf{w}$	$(d+1) \times 1$
Vector điểm số	$\mathbf{s} = \mathbf{X}\mathbf{w}$	$n \times 1$
Vector nhãn dự đoán	$\hat{\mathbf{y}} = \text{sign}(\mathbf{s})$	$n \times 1$
Vector nhãn thực	$\mathbf{y}$	$n \times 1$
Vector margin	$\mathbf{m} = \mathbf{y} \odot \mathbf{s}$	$n \times 1$

Ví dụ số với dữ liệu sinh viên

Quay lại ma trận $\mathbf{X}$ và vector $\mathbf{y}$ của ví dụ ở mục 1, với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$. Ta đã tính được

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} -4{,}5 \\ -2{,}5 \\ -3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix} \quad \Rightarrow \quad \hat{\mathbf{y}} = \text{sign}(\mathbf{s}) = \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} 4{,}5 \\ 2{,}5 \\ 3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix}$$

Vector nhãn dự đoán $\hat{\mathbf{y}}$ trùng với $\mathbf{y}$ và mọi margin đều dương, nghĩa là mô hình phân loại đúng tất cả các mẫu.

Bây giờ thử một bộ tham số chưa tốt: $\mathbf{w} = [-2{,}5 \quad 1 \quad 0]^T$. Mô hình này bỏ qua hoàn toàn số buổi vắng và chỉ quan tâm đến số giờ tự học, với $s = -2{,}5 + x_1$. Khi đó

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} -0{,}5 \\ 0{,}5 \\ 1{,}5 \\ 3{,}5 \\ 5{,}5 \\ 6{,}5 \end{bmatrix} \quad \Rightarrow \quad \hat{\mathbf{y}} = \begin{bmatrix} -1 \\ +1 \\ +1 \\ +1 \\ +1 \\ +1 \end{bmatrix}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} 0{,}5 \\ -0{,}5 \\ -1{,}5 \\ 3{,}5 \\ 5{,}5 \\ 6{,}5 \end{bmatrix}$$

Hai margin âm ở vị trí 2 và 3 cho biết sinh viên 2 và 3 bị phân loại sai. Với cả hai mẫu, $y_i = -1$ nhưng $\hat{y}_i = +1$, tức là lỗi **báo nhầm** (thực tế rớt nhưng bị dự đoán đậu). Như vậy, vector margin **m** cho biết **mẫu nào sai** và **sai nặng đến đâu**. Đây sẽ là thông tin then chốt để xây dựng hàm mất mát và cập nhật tham số ở mục 3.

2.3. Đường biên quyết định và cách mô hình phân chia không gian đặc trưng

Đường biên quyết định

Perceptron gán nhãn dựa vào dấu của **xw**. Do đó, ranh giới giữa hai nhãn là tập hợp các điểm có điểm số bằng đúng 0:

$$\{\mathbf{x} : \mathbf{x}\mathbf{w} = 0\} \quad \Leftrightarrow \quad w_0 + w_1 x_1 + \cdots + w_d x_d = 0$$

Tập hợp này gọi là **đường biên quyết định (decision boundary)**. Nó chia không gian đặc trưng thành **hai nửa không gian**:

- một nửa có $\mathbf{x}\mathbf{w} \geq 0$ (gồm cả biên) và mô hình dự đoán $\hat{y} = +1$;
- một nửa có $\mathbf{x}\mathbf{w} < 0$ và mô hình dự đoán $\hat{y} = -1$.

Hình dạng của đường biên phụ thuộc vào số đặc trưng d:

Số đặc trưng d	Đường biên là	Phương trình
1	Một **điểm** trên trục số	$w_0 + w_1 x_1 = 0$
2	Một **đường thẳng** trên mặt phẳng	$w_0 + w_1 x_1 + w_2 x_2 = 0$
3	Một **mặt phẳng** trong không gian	$w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 = 0$
> 3	Một **siêu phẳng**	$\mathbf{x}\mathbf{w} = 0$

Dù số chiều là bao nhiêu, đường biên của Perceptron luôn là một đối tượng **"phẳng"**. Vì vậy Perceptron được gọi là **bộ phân loại tuyến tính (linear classifier)**.

Tìm phương trình đường biên cụ thể

Trường hợp $d = 1$. Giải $w_0 + w_1 x_1 = 0$, ta có điểm biên

$$x_1 = -\frac{w_0}{w_1} \qquad (w_1 \neq 0)$$

Ví dụ ở mục 1.2, với 8 sinh viên học 1,2, ... ,8 giờ, thử $\mathbf{w} = [-4{,}5 \quad 1]^T$ thì điểm biên là $x_1 = 4{,}5$. Với mô hình này, các sinh viên học từ 5 giờ trở lên có $s > 0$ nên được dự đoán đậu, và nếu có thêm sinh viên học 20 giờ thì $s = 15{,}5 > 0$ cũng được dự đoán đậu. Khác với hồi quy tuyến tính ở mục 1.2, mẫu "rất đúng" này **không làm lệch đường biên**, vì mô hình chỉ quan tâm điểm nằm về phía nào chứ không cố đưa s về đúng $+1$.

Trường hợp $d = 2$. Nếu $w_2 \neq 0$, ta rút x_2 theo x_1:

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2} x_1$$

Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$, ta có

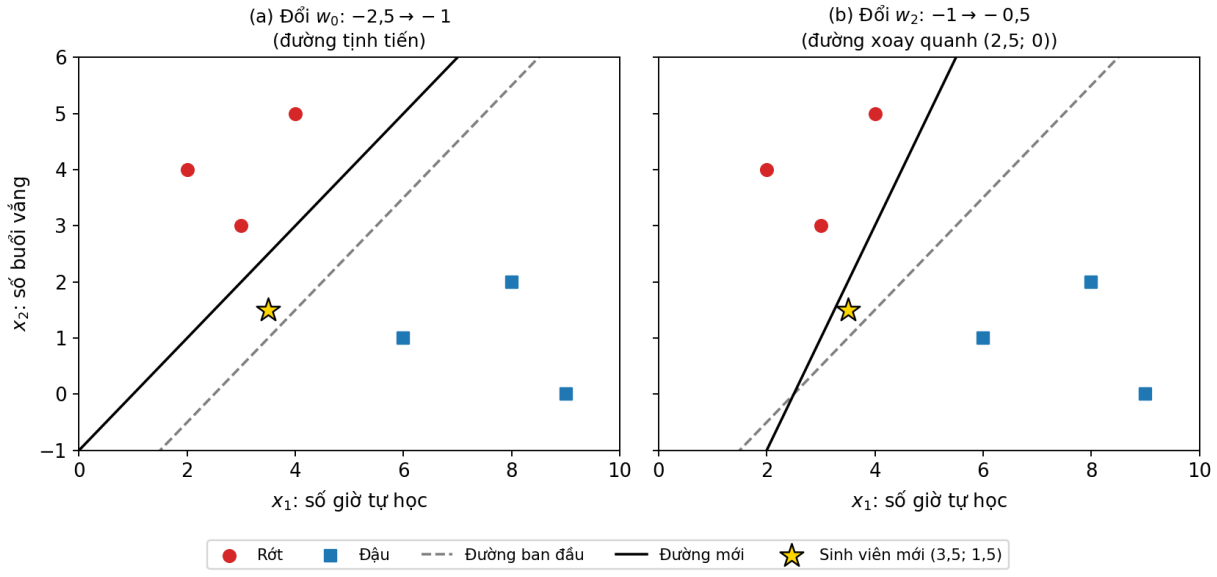
$$x_2 = -\frac{-2{,}5}{-1} - \frac{1}{-1} x_1 = x_1 - 2{,}5$$

đúng là đường thẳng đã vẽ ở Hình 2.

Vai trò của w_0 và các trọng số $w_1, \ldots, w_d$

Hai nhóm tham số điều khiển đường biên theo hai cách khác nhau. Ta quan sát bằng cách thay đổi từng tham số của bộ $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ (Hình 5):

- **Thay đổi hệ số chệch** w_0 làm đường biên **tịnh tiến** song song với chính nó. Khi đổi w_0 từ $-2{,}5$ sang -1, đường biên trở thành $x_2 = x_1 - 1$.
- **Thay đổi trọng số** w_j làm đường biên **xoay**. Khi đổi w_2 từ -1 sang $-0{,}5$, đường biên trở thành $x_2 = 2x_1 - 5$ và xoay quanh điểm $(2{,}5;\ 0)$.



Hình 5. Ảnh hưởng của từng tham số đến đường biên. Ngôi sao là một sinh viên mới (học 3,5 giờ, vắng 1,5 buổi) nằm về các phía khác nhau của ba đường biên.

Cả hai bộ tham số mới vẫn phân loại đúng 6 sinh viên ban đầu, nhưng chúng cho kết luận **khác nhau** với sinh viên mới $\mathbf{x} = [1 \quad 3{,}5 \quad 1{,}5]$:

Bộ tham số $\mathbf{w}$	Điểm số s	Nhãn dự đoán
$[-2{,}5 \quad 1 \quad -1]^T$ (ban đầu)	$-0{,}5$	-1 (rớt)
$[-1 \quad 1 \quad -1]^T$ (đổi w_0)	1	$+1$ (đậu)
$[-2{,}5 \quad 1 \quad -0{,}5]^T$ (đổi w_2)	$0{,}25$	$+1$ (đậu)

Điều này cho thấy: nhiều bộ tham số cùng đúng trên dữ liệu huấn luyện vẫn có thể **dự đoán khác nhau trên dữ liệu mới**. Việc chọn $\mathbf{w}$ như thế nào là rất quan trọng, và ta sẽ quay lại vấn đề này khi phân tích cơ chế học của Perceptron.

Nhân tham số với một hằng số dương không đổi bộ phân loại

Có một tính chất đơn giản nhưng sẽ rất hữu ích về sau. Với mọi hằng số $c > 0$, vector $c\,\mathbf{w}$ cho cùng một đường biên và cùng nhãn dự đoán với $\mathbf{w}$:

$$\text{sign}\big(\mathbf{x}\,(c\,\mathbf{w})\big) = \text{sign}(c \cdot \mathbf{xw}) = \text{sign}(\mathbf{xw}) \qquad (c > 0)$$

vì nhân s với một số dương không làm đổi dấu của s. Ví dụ, $[-5 \quad 2 \quad -2]^T$ và $[-2{,}5 \quad 1 \quad -1]^T$ định nghĩa **cùng một bộ phân loại**. Như vậy, điều quan trọng của $\mathbf{w}$ không phải độ lớn tuyệt đối mà là **hướng** của nó (và vị trí tương đối giữa w_0 với các trọng số). Lưu ý rằng margin m_i thì **có đổi**: nó được nhân với c.

Giới hạn: đường biên luôn là đường phẳng

Vì đường biên của Perceptron là một siêu phẳng, mô hình chỉ có thể phân loại hoàn hảo những dữ liệu **phân tách tuyến tính được**, tức là tồn tại $\mathbf{w}$ để $y_i\,\mathbf{x}_i\mathbf{w} > 0$ với mọi i. Với dữ liệu mà hai lớp đan xen nhau (như bài toán XOR ở mục 4), không có đường thẳng nào làm đúng cả, và Perceptron chắc chắn sẽ mắc sai sót. Đây là giới hạn cơ bản của mô hình, và cũng là động lực để phát triển các mô hình mạnh hơn như mạng nơ-ron nhiều lớp.

Tóm tắt mục 2

- Perceptron tính **điểm số** $s = \mathbf{xw} = w_0 + w_1x_1 + \cdots + w_dx_d$, rồi dùng **hàm dấu** để chuyển điểm số thành nhãn: $\hat{y} = \text{sign}(\mathbf{xw})$, với $\text{sign}(s) = +1$ khi $s \geq 0$ và $\text{sign}(s) = -1$ khi $s < 0$.
- Có thể nhớ gọn: **Perceptron = hồi quy tuyến tính + hàm dấu**. Hàm dấu có đạo hàm bằng 0 hầu như mọi nơi nên cần một cách huấn luyện khác với Gradient Descent trên MSE.
- Với toàn bộ dữ liệu, ta viết $\hat{\mathbf{y}} = \text{sign}(\mathbf{Xw})$, trong đó hàm dấu được áp dụng lên từng phần tử. **Margin** $m_i = y_i s_i$ cho biết mẫu i đúng hay sai ($m_i > 0$ là đúng, $m_i < 0$ là sai) và mức độ đúng sai.
- **Đường biên quyết định** là siêu phẳng $\mathbf{xw} = 0$. Thay đổi w_0 làm đường biên **tịnh tiến**, thay đổi các trọng số làm đường biên **xoay**. Nhân $\mathbf{w}$ với hằng số dương không đổi bộ phân loại.
- Perceptron chỉ phân loại hoàn hảo được dữ liệu **phân tách tuyến tính được**.

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ và $\mathbf{x} = [1 \quad 6 \quad 3]$, hãy tính s và $\hat{y}$.
2. Với cùng $\mathbf{w}$ và $\mathbf{x} = [1 \quad 4 \quad 1{,}5]$, hãy tính s và $\hat{y}$. Kết quả này phụ thuộc vào quy ước nào?
3. Cho $\mathbf{w} = [-6 \quad 2 \quad 3]^T$. Hãy viết phương trình đường biên dưới dạng x_2 theo x_1. Gốc tọa độ $(0{,}0)$ nằm ở phía được dự đoán nhãn nào?
4. Với $\mathbf{w} = [-2{,}5 \quad 0 \quad -1]^T$ trên 6 sinh viên của ví dụ, hãy tính $\hat{\mathbf{y}}$ và $\mathbf{m}$. Có bao nhiêu mẫu bị phân loại sai?
5. Vì sao ta không thể huấn luyện Perceptron bằng cách áp dụng trực tiếp Gradient Descent lên MSE giữa y và $\hat{y} = \text{sign}(\mathbf{xw})$?

Gợi ý đáp án

1. $s = -2{,}5 + 6 - 3 = 0{,}5 \geq 0$ nên $\hat{y} = +1$.
2. $s = -2{,}5 + 4 - 1{,}5 = 0$. Điểm nằm đúng trên đường biên nên $\hat{y}$ phụ thuộc quy ước tại $s = 0$. Theo quy ước của bài giảng, $\hat{y} = +1$.
3. $x_2 = -\frac{-6}{3} - \frac{2}{3}x_1 = 2 - \frac{2}{3}x_1$. Tại $(0{,}0)$, $s = -6 < 0$ nên gốc tọa độ thuộc phía $\hat{y} = -1$.
4. $s = -2{,}5 - x_2 = [-6{,}5 \quad -5{,}5 \quad -7{,}5 \quad -3{,}5 \quad -4{,}5 \quad -2{,}5]^T$, tất cả đều âm nên $\hat{\mathbf{y}} = [-1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1]^T$. Margin $\mathbf{m} = \mathbf{y} \odot \mathbf{s} = [6{,}5 \quad 5{,}5 \quad 7{,}5 \quad -3{,}5 \quad -4{,}5 \quad -2{,}5]^T$. Có 3 margin âm nên có 3 mẫu sai (sinh viên 4,5,6), đều là lỗi **bỏ sót** ($y_i = +1$ nhưng $\hat{y}_i = -1$).
5. Hàm dấu là hàm hằng ở hai phía của $s = 0$ nên đạo hàm bằng 0 tại mọi $s \neq 0$. Do đó gradient của hàm mất mát theo $\mathbf{w}$ bằng 0 hầu như mọi nơi, và Gradient Descent không có hướng để cập nhật tham số.

3. Huấn luyện mô hình Perceptron

Ở mục 2, ta đã có mô hình $\hat{y} = \text{sign}(\mathbf{xw})$. Tuy nhiên, mô hình này mới chỉ là một "chiếc máy" chưa được cài đặt: vector tham số $\mathbf{w}$ vẫn chưa biết. Ta đã thấy rằng có vô số $\mathbf{w}$ phân loại đúng dữ liệu, nhưng làm sao để máy tính **tự tìm ra** một $\mathbf{w}$ như vậy từ dữ liệu? Đó là bài toán **huấn luyện (training)**.

Trong mục này, ta trả lời ba câu hỏi, bám theo khung huấn luyện đã học ở bài hồi quy tuyến tính:

- Làm thế nào để **đo mức độ sai** của một bộ tham số? (hàm mất mát)
- Khi gặp một mẫu bị phân loại sai, **nên chỉnh $\mathbf{w}$ theo hướng nào**? (quy tắc cập nhật)
- Lặp lại việc chỉnh sửa ra sao, và **khi nào thì dừng**? (thuật toán)

Ta tiếp tục dùng quy ước ký hiệu ở các mục trước: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn $y_i, \hat{y}_i \in \{-1, +1\}$, điểm số $s_i = \mathbf{x}_i\mathbf{w}$ và margin $m_i = y_i s_i$.

3.1. Nhắc lại khung huấn luyện: mô hình, hàm mất mát và thuật toán tối ưu

Ở bài hồi quy tuyến tính, quá trình huấn luyện gồm ba thành phần. Ta đặt Perceptron cạnh hồi quy tuyến tính để thấy thành phần nào đã có, thành phần nào còn thiếu:

Thành phần	Hồi quy tuyến tính	Perceptron
Mô hình	$\hat{y} = \mathbf{xw}$	$\hat{y} = \text{sign}(\mathbf{xw})$ (đã có ở mục 2)
Hàm mất mát	MSE	**Hàm mất mát Perceptron** (mục 3.2)
Thuật toán tối ưu	Gradient Descent	**Gradient Descent từng mẫu** (mục 3.3 và 3.4)

Mục tiêu huấn luyện vẫn như cũ: **tìm $\mathbf{w}$ làm hàm mất mát nhỏ nhất**.

Câu hỏi tự nhiên là: hàm mất mát nên là gì? Điều ta thực sự quan tâm là **số mẫu bị phân loại sai**. Ta có thể định nghĩa "mất mát 0–1" của một mẫu bằng 1 nếu bị sai và 0 nếu đúng. Tuy nhiên, như đã chỉ ra ở mục 2.1, hàm này **không dùng được với Gradient Descent**: nó là hàm hằng từng khúc nên gradient bằng 0 hầu như ở mọi nơi, và thuật toán không biết phải chỉnh $\mathbf{w}$ theo hướng nào.

Vì vậy, ta cần một hàm mất mát **vẫn phản ánh đúng việc đúng hay sai**, nhưng **có độ dốc** để chỉ hướng đi cho thuật toán. Công cụ phù hợp chính là **margin** $m_i = y_i s_i$ đã gặp ở mục 2.

3.2. Hàm mất mát của Perceptron

Ý tưởng: chỉ phạt mẫu sai, và sai càng nặng thì phạt càng nhiều

Nhắc lại ý nghĩa của margin $m_i = y_i s_i = y_i(\mathbf{x}_i\mathbf{w})$:

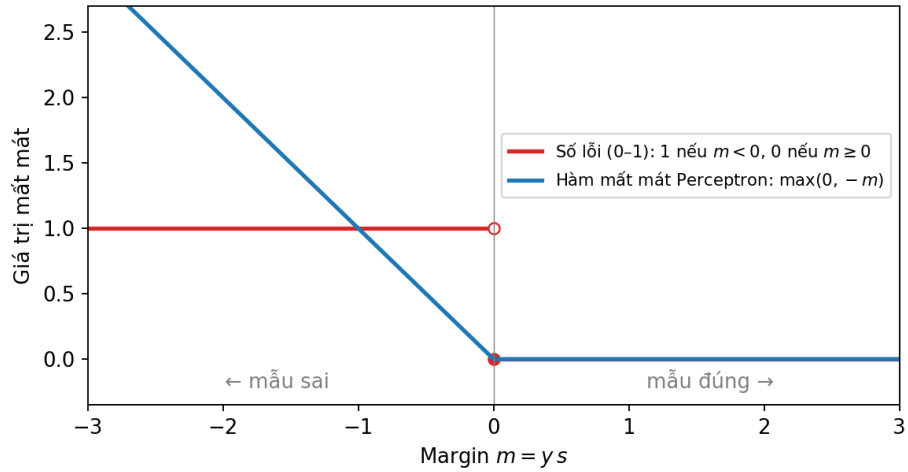
- $m_i > 0$: mẫu được phân loại **đúng**. Ta không cần phạt.

- $m_i < 0$: mẫu bị phân loại **sai**. Ta phạt, và $|m_i|$ càng lớn (mẫu nằm càng sâu trong vùng sai) thì phạt càng nặng.

Từ đó, ta định nghĩa **mất mát Perceptron** của mẫu thứ i:

$$\ell_i(\mathbf{w}) = \max(0,\ -y_i\,\mathbf{x}_i\mathbf{w}) = \max(0,\ -m_i) = \begin{cases} 0 & \text{nếu } m_i \geq 0 \\ -m_i & \text{nếu } m_i < 0 \end{cases}$$

Hình 6 so sánh hàm mất mát này với số lỗi 0–1. Hai hàm đều bằng 0 khi mẫu đúng. Khi mẫu sai, số lỗi luôn bằng 1 bất kể sai nhiều hay ít, còn mất mát Perceptron **tăng dần theo mức độ sai** và có **độ dốc khác** 0, nhờ đó ta có thể dùng đạo hàm.



Hình 6. So sánh số lỗi 0–1 *và mất mát Perceptron theo margin* $m = y\,s$. *Mất mát Perceptron bằng* 0 *với mẫu đúng và tăng tuyến tính theo mức độ sai.*

Với toàn bộ tập dữ liệu, **hàm mất mát tổng** là tổng mất mát của từng mẫu:

$$J(\mathbf{w}) = \sum_{i=1}^{n} \max\,(0,\ -y_i\,\mathbf{x}_i\mathbf{w}) = \sum_{i \in \mathcal{M}(\mathbf{w})} (-y_i\,\mathbf{x}_i\mathbf{w})$$

Trong đó $\mathcal{M}(\mathbf{w}) = \{i : y_i\,\mathbf{x}_i\mathbf{w} \leq 0\}$ là **tập các mẫu bị sai** (hoặc chưa đúng chắc chắn) với bộ tham số $\mathbf{w}$ hiện tại. Nói cách khác, $J(\mathbf{w})$ chỉ cộng dồn phần "lỗi" của những mẫu sai.

Một số tính chất cần nhớ:

- $J(\mathbf{w}) \geq 0$ với mọi $\mathbf{w}$.
- $J(\mathbf{w}) = 0$ khi mọi mẫu đều có $m_i \geq 0$, tức là không còn mẫu nào nằm sai phía.
- Mỗi mẫu sai đóng góp một lượng bằng $|m_i| = |s_i|$. Vì khoảng cách từ mẫu đến đường biên là $|s_i|/\|\,\tilde{\mathbf{w}}\,\|$, ta có thể nói $J(\mathbf{w})$ tỉ lệ với **tổng khoảng cách từ các mẫu sai đến đường biên**. Mẫu sai nằm càng xa đường biên thì càng bị phạt nặng.

Lưu ý về nghiệm tầm thường. Với $\mathbf{w} = \mathbf{0}$, mọi margin đều bằng 0 nên $J(\mathbf{0}) = 0$. Tuy nhiên đây là nghiệm vô nghĩa, vì mô hình chưa phân biệt được gì. Để thuật toán

không "tưởng" rằng $\mathbf{w} = \mathbf{0}$ đã tốt, ta xem mẫu có $m_i \le 0$ (kể cả $m_i = 0$) là mẫu **cần cập nhật**. Đó là lý do tập $\mathcal{M}(\mathbf{w})$ ở trên dùng dấu $\le$.

Ví dụ số với dữ liệu sinh viên

Ta tính $J(\mathbf{w})$ cho ba bộ tham số đã gặp ở mục 2 trên 6 sinh viên:

Bộ tham số $\mathbf{w}$	Vector margin $\mathbf{m}$	Mẫu sai	$J(\mathbf{w})$
$[-2{,}5 \quad 1 \quad -1]^T$	$[4{,}5 \quad 2{,}5 \quad 3{,}5 \quad 2{,}5 \quad 3{,}5 \quad 6{,}5]^T$	Không có	0
$[-2{,}5 \quad 1 \quad 0]^T$	$[0{,}5 \quad -0{,}5 \quad -1{,}5 \quad 3{,}5 \quad 5{,}5 \quad 6{,}5]^T$	Sinh viên 2,3	$0{,}5 + 1{,}5 = 2$
$[-2{,}5 \quad 0 \quad -1]^T$	$[6{,}5 \quad 5{,}5 \quad 7{,}5 \quad -3{,}5 \quad -4{,}5 \quad -2{,}5]^T$	Sinh viên 4,5,6	$3{,}5 + 4{,}5 + 2{,}5 = 10{,}5$

Bộ tham số càng tệ thì J càng lớn, và $J = 0$ ứng với bộ tham số phân loại đúng toàn bộ. Đây đúng là hành vi ta mong muốn ở một hàm mất mát.

3.3. Quy tắc cập nhật khi phân loại sai

Gradient của mất mát một mẫu

Ta tính gradient của $\ell_i(\mathbf{w})$ theo $\mathbf{w}$ trong hai trường hợp:

- Nếu mẫu i **đúng** ($m_i > 0$): $\ell_i = 0$ là hằng số nên gradient bằng vector $\mathbf{0}$. Mẫu này **không ảnh hưởng** đến việc cập nhật.
- Nếu mẫu i **sai** ($m_i \le 0$): $\ell_i = -y_i\,\mathbf{x}_i\mathbf{w}$ là hàm tuyến tính của $\mathbf{w}$, nên gradient là

$$\nabla \ell_i(\mathbf{w}) = -y_i\,\mathbf{x}_i^T$$

Ở đây ta dùng $\mathbf{x}_i^T$ (vector cột) để gradient có cùng kích thước $(d+1) \times 1$ với $\mathbf{w}$. Riêng tại $m_i = 0$, hàm $\max(0, -m)$ có "góc nhọn" nên đạo hàm không xác định, và ta lấy $-y_i\mathbf{x}_i^T$ làm hướng cập nhật.

Quy tắc cập nhật

Áp dụng Gradient Descent với **một mẫu** tại mỗi bước, tức là $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i(\mathbf{w})$, ta được **quy tắc cập nhật Perceptron**:

$$\mathbf{w} \leftarrow \begin{cases} \mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T & \text{nếu } y_i\,\mathbf{x}_i\mathbf{w} \le 0 \\ \mathbf{w} & \text{nếu } y_i\,\mathbf{x}_i\mathbf{w} > 0 \end{cases}$$

Trong đó $\eta > 0$ là **tốc độ học (learning rate)**. Viết theo từng thành phần, quy tắc trên là

$$w_0 \leftarrow w_0 + \eta\, y_i, \qquad w_j \leftarrow w_j + \eta\, y_i\, x_{ij} \quad (j = 1, \dots, d)$$

Vì $y_i \in \{-1, +1\}$, quy tắc này có thể phát biểu thành hai trường hợp rất dễ nhớ:

Mẫu bị sai	Loại sai	Cập nhật	Ý nghĩa
$y_i = +1$	Bỏ sót (dự đoán nhầm là -1)	$\mathbf{w} \leftarrow \mathbf{w} + \eta\, \mathbf{x}_i^T$	**Cộng** vector đặc trưng vào $\mathbf{w}$: kéo $\mathbf{w}$ lại gần $\mathbf{x}_i$
$y_i = -1$	Báo nhầm (dự đoán nhầm là $+1$)	$\mathbf{w} \leftarrow \mathbf{w} - \eta\, \mathbf{x}_i^T$	**Trừ** vector đặc trưng khỏi $\mathbf{w}$: đẩy $\mathbf{w}$ ra xa $\mathbf{x}_i$

Ví dụ. Với $\mathbf{w} = \mathbf{0}$, $\eta = 1$ và sinh viên 1 ($\mathbf{x}_1 = [1 \quad 2 \quad 4]$, $y_1 = -1$), ta có $m_1 = 0 \leq 0$ nên cần cập nhật:

$$\mathbf{w} \leftarrow \begin{bmatrix}0\\0\\0\end{bmatrix} + 1 \cdot (-1) \cdot \begin{bmatrix}1\\2\\4\end{bmatrix} = \begin{bmatrix}-1\\-2\\-4\end{bmatrix}$$

Sinh viên này thuộc lớp rớt nên $\mathbf{w}$ bị "đẩy ra xa" vector đặc trưng của sinh viên đó: các trọng số của giờ học và buổi vắng đều giảm.

Vì sao quy tắc này hợp lý?

Ta kiểm tra điều gì xảy ra với **chính mẫu vừa được cập nhật**. Sau khi cập nhật, margin mới của mẫu i là

$$m_i' = y_i\, \mathbf{x}_i(\mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T) = y_i\, \mathbf{x}_i\mathbf{w} + \eta\, y_i^2\, \mathbf{x}_i\mathbf{x}_i^T = m_i + \eta\ \|\, \mathbf{x}_i\, \|^2$$

Ta đã dùng $y_i^2 = 1$ (vì $y_i = \pm 1$) và $\mathbf{x}_i\mathbf{x}_i^T = \|\, \mathbf{x}_i\, \|^2$. Vì $\|\, \mathbf{x}_i\, \|^2 > 0$ (phần tử đầu tiên của $\mathbf{x}_i$ luôn bằng 1), **margin của mẫu đó luôn tăng thêm đúng** $\eta\, \|\, \mathbf{x}_i\, \|^2$. Như vậy, mỗi lần cập nhật đều đẩy mẫu sai đang xét **về phía đúng hơn**. Trong ví dụ trên, $\|\, \mathbf{x}_1\, \|^2 = 1 + 4 + 16 = 21$ nên margin của sinh viên 1 tăng từ 0 lên 21, đúng như điểm số $s_1 = -21$ sau cập nhật.

Cần lưu ý rằng một lần cập nhật chỉ **cải thiện mẫu đang xét**, chưa chắc sửa được hết lỗi, và có thể **làm xấu đi** các mẫu khác. Đó là lý do ta phải lặp lại nhiều lần, như ở mục 3.4.

So sánh với Gradient Descent ở hồi quy tuyến tính

Cả hai đều có dạng $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot$ (gradient), nhưng khác nhau ở hai điểm:

- Ở hồi quy tuyến tính, mọi mẫu đều đóng góp vào gradient, với độ lớn tỉ lệ với sai số liên tục. Ở Perceptron, **chỉ những mẫu sai mới đóng góp**, và mức đóng góp luôn là $\pm\eta\, \mathbf{x}_i^T$.
- Gradient Descent ở hồi quy thường duyệt toàn bộ dữ liệu rồi mới cập nhật một lần. Perceptron cập nhật **ngay sau mỗi mẫu**. Cách làm này gọi là **Gradient Descent ngẫu nhiên (stochastic gradient descent, SGD)** khi mỗi lần chỉ dùng một mẫu.

Dạng ma trận (cập nhật theo toàn bộ dữ liệu)

Nếu muốn gom tất cả mẫu sai trong một lần, ta tính gradient của $J(\mathbf{w})$. Đặt vector $\mathbf{u} \in \mathbb{R}^n$ với $u_i = y_i$ nếu $i \in \mathcal{M}(\mathbf{w})$ và $u_i = 0$ nếu ngược lại. Khi đó

$$\nabla J(\mathbf{w}) = -\sum_{i \in \mathcal{M}(\mathbf{w})} y_i\, \mathbf{x}_i^T = -\mathbf{X}^T\mathbf{u}, \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta\, \mathbf{X}^T\mathbf{u}$$

Về kích thước, $\mathbf{X}^T$ có kích thước $(d+1) \times n$ và $\mathbf{u}$ có kích thước $n \times 1$ nên $\mathbf{X}^T\mathbf{u}$ có kích thước $(d+1) \times 1$, khớp với $\mathbf{w}$. Trong bài giảng này, ta dùng dạng **từng mẫu** vì nó là thuật toán Perceptron kinh điển và dễ tính tay hơn.

3.4. Thuật toán Perceptron qua từng bước

Một số khái niệm

- Một lần **duyệt qua toàn bộ n mẫu** một lượt được gọi là một **epoch**.
- Trong mỗi epoch, ta lần lượt xét từng mẫu. Nếu mẫu cần cập nhật ($m_i \le 0$), ta cập nhật $\mathbf{w}$ **ngay lập tức**. Bộ $\mathbf{w}$ mới được dùng luôn cho các mẫu tiếp theo trong cùng epoch.

Thuật toán

Đầu vào: ma trận đặc trưng $\mathbf{X}$ (kích thước $n \times (d+1)$), vector nhãn $\mathbf{y}$ ($n \times 1$, các phần tử ± 1), tốc độ học $\eta > 0$, số epoch tối đa $T_{\max}$.

Đầu ra: vector tham số $\mathbf{w}$.

1. **Khởi tạo** $\mathbf{w} = \mathbf{0}$ (vector cột kích thước $(d+1) \times 1$).
2. **Lặp** cho $t = 1, 2, \ldots, T_{\max}$ (mỗi lần lặp là một epoch):
    1. Đặt bộ đếm số lần cập nhật $c = 0$.
    2. **Với mỗi** mẫu $i = 1, \ldots, n$ (theo thứ tự cố định hoặc đã xáo trộn):
        1. Tính điểm số $s_i = \mathbf{x}_i\mathbf{w}$ và margin $m_i = y_i s_i$.
        2. **Nếu** $m_i \le 0$: cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T$ và tăng c lên 1.
    3. **Nếu** $c = 0$ (cả epoch không có lần cập nhật nào): **dừng**.
3. **Trả về** $\mathbf{w}$.

Điều kiện dừng

Thuật toán dừng khi xảy ra một trong hai điều kiện:

- **Một epoch trọn vẹn không có lần cập nhật nào** ($c = 0$). Khi đó mọi mẫu đều có $m_i > 0$, tức là đều được phân loại đúng, và vì $\mathbf{w}$ không đổi nên các epoch sau cũng sẽ không đổi. Thuật toán đã **hội tụ**.
- **Đạt số epoch tối đa** $T_{\max}$. Điều kiện này là "van an toàn", cần thiết khi dữ liệu không phân tách tuyến tính được. Khi đó luôn có mẫu sai nên $c = 0$ không bao giờ xảy ra, và nếu không có $T_{\max}$ thì thuật toán chạy mãi. Ta sẽ phân tích kỹ ở mục 4.

Lưu ý. Ta phải duyệt thêm một epoch hoàn toàn "sạch" mới biết thuật toán đã hội tụ. Vì vậy, epoch cuối cùng thường không có lần cập nhật nào, và chỉ có tác dụng xác nhận.

3.5. Ví dụ tính tay một vòng huấn luyện

Ta chạy thuật toán trên dữ liệu 6 sinh viên của mục 1, với:

- khởi tạo $\mathbf{w} = [0 \quad 0 \quad 0]^T$;

- tốc độ học $\eta = 1$;
- duyệt các mẫu theo thứ tự $1 \to 2 \to \cdots \to 6$.

Nhắc lại các vector đặc trưng và nhãn:

Sinh viên	$\mathbf{x}_i$	y_i
1	$[1 \quad 2 \quad 4]$	-1
2	$[1 \quad 3 \quad 3]$	-1
3	$[1 \quad 4 \quad 5]$	-1
4	$[1 \quad 6 \quad 1]$	$+1$
5	$[1 \quad 8 \quad 2]$	$+1$
6	$[1 \quad 9 \quad 0]$	$+1$

Epoch 1

Bắt đầu với $\mathbf{w}^{(0)} = [0 \quad 0 \quad 0]^T$.

Mẫu 1. $s_1 = \mathbf{x}_1\mathbf{w} = 0$, nên $m_1 = (-1) \cdot 0 = 0 \leq 0$. Cần cập nhật:

$$\mathbf{w} \leftarrow \begin{bmatrix}0\\0\\0\end{bmatrix} + (-1)\begin{bmatrix}1\\2\\4\end{bmatrix} = \begin{bmatrix}-1\\-2\\-4\end{bmatrix}$$

Mẫu 2. $s_2 = -1 - 6 - 12 = -19$, nên $m_2 = (-1)(-19) = 19 > 0$. Mẫu đúng, **giữ nguyên w**.

Mẫu 3. $s_3 = -1 - 8 - 20 = -29$, nên $m_3 = 29 > 0$. Mẫu đúng, **giữ nguyên**.

Mẫu 4. $s_4 = -1 - 12 - 4 = -17$, nên $m_4 = (+1)(-17) = -17 \leq 0$. Mẫu này bị **sai** (thực tế đậu nhưng mô hình dự đoán rớt). Cập nhật:

$$\mathbf{w} \leftarrow \begin{bmatrix}-1\\-2\\-4\end{bmatrix} + (+1)\begin{bmatrix}1\\6\\1\end{bmatrix} = \begin{bmatrix}0\\4\\-3\end{bmatrix}$$

Kiểm tra: margin của mẫu 4 tăng từ -17 lên $-17 + \| \mathbf{x}_4 \|^2 = -17 + 38 = 21$, khớp với $s_4 = 24 - 3 = 21$ sau cập nhật.

Mẫu 5. $s_5 = 0 + 32 - 6 = 26$, nên $m_5 = 26 > 0$. **Giữ nguyên**.

Mẫu 6. $s_6 = 0 + 36 + 0 = 36$, nên $m_6 = 36 > 0$. **Giữ nguyên**.

Kết thúc epoch 1 có $c = 2$ lần cập nhật và $\mathbf{w} = [0 \quad 4 \quad -3]^T$. Nếu kiểm tra lại toàn bộ dữ liệu với bộ $\mathbf{w}$ này, ta có $\mathbf{s} = [-4 \quad 3 \quad 1 \quad 21 \quad 26 \quad 36]^T$ và $\mathbf{m} = [4 \quad -3 \quad -1 \quad 21 \quad 26 \quad 36]^T$, nên **sinh viên 2 và 3 đang bị phân loại sai**. Vì vậy ta phải tiếp tục.

Các epoch tiếp theo

Việc tính toán ở các epoch sau hoàn toàn tương tự. Để gọn, ta chỉ liệt kê các lần **có cập nhật**:

Lần cập nhật	Epoch	Mẫu	s_i trước cập nhật	m_i	**w** sau cập nhật
1	1	1	0	0	$[-1 \quad -2 \quad -4]^T$
2	1	4	-17	-17	$[0 \quad 4 \quad -3]^T$
3	2	2	3	-3	$[-1 \quad 1 \quad -6]^T$
4	2	4	-1	-1	$[0 \quad 7 \quad -5]^T$
5	3	2	6	-6	$[-1 \quad 4 \quad -8]^T$

Cụ thể:

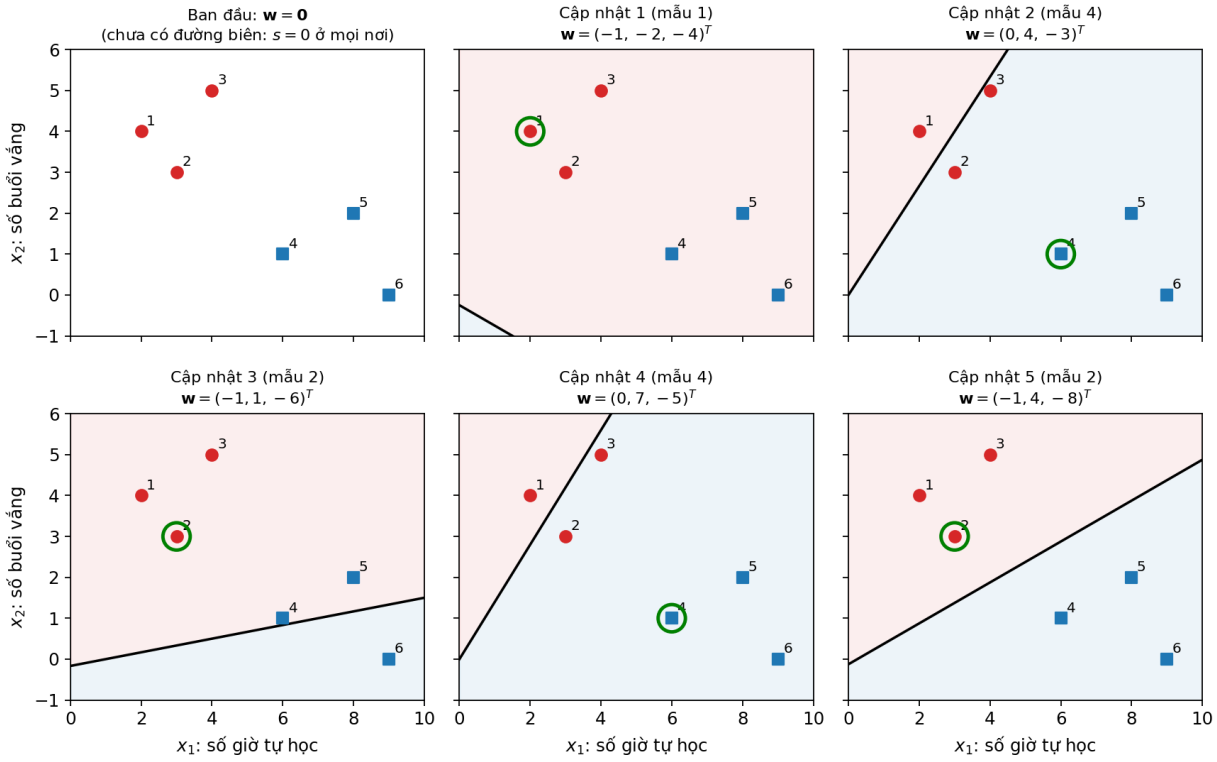
- **Epoch 2** (bắt đầu với $\mathbf{w} = [0 \quad 4 \quad -3]^T$): mẫu 1 đúng ($s_1 = -4$); mẫu 2 **sai** ($s_2 = 3 > 0$ trong khi $y_2 = -1$) nên cập nhật $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}_2^T = [-1 \quad 1 \quad -6]^T$; mẫu 3 đúng ($s_3 = -27$); mẫu 4 **sai** ($s_4 = -1 < 0$ trong khi $y_4 = +1$) nên cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}_4^T = [0 \quad 7 \quad -5]^T$; mẫu 5 và 6 đúng ($s_5 = 46$, $s_6 = 63$). Cuối epoch 2, sinh viên 2 và 3 vẫn đang sai.
- **Epoch 3** (bắt đầu với $\mathbf{w} = [0 \quad 7 \quad -5]^T$): mẫu 1 đúng ($s_1 = -6$); mẫu 2 **sai** ($s_2 = 6 > 0$) nên $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}_2^T = [-1 \quad 4 \quad -8]^T$; các mẫu 3 đến 6 đều đúng với bộ **w** mới ($s_3 = -25$, $s_4 = 15$, $s_5 = 15$, $s_6 = 35$).
- **Epoch 4** (bắt đầu với $\mathbf{w} = [-1 \quad 4 \quad -8]^T$): không có lần cập nhật nào ($c = 0$), nên **thuật toán dừng**. Vector điểm số và margin lúc này là

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} -25 \\ -13 \\ -25 \\ 15 \\ 15 \\ 35 \end{bmatrix}, \qquad \hat{\mathbf{y}} = \text{sign}(\mathbf{s}) = \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix} = \mathbf{y}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} 25 \\ 13 \\ 25 \\ 15 \\ 15 \\ 35 \end{bmatrix}$$

Tổng cộng thuật toán thực hiện 5 lần cập nhật trong 3 epoch đầu, và epoch thứ 4 dùng để xác nhận. Vector tham số tìm được là $\mathbf{w} = [-1 \quad 4 \quad -8]^T$, tương ứng với đường biên

$$-1 + 4x_1 - 8x_2 = 0 \quad \Leftrightarrow \quad x_2 = \frac{x_1}{2} - \frac{1}{8}$$

Hình 7 cho thấy đường biên dịch chuyển sau mỗi lần cập nhật.



Hình 7. Đường biên quyết định sau mỗi lần cập nhật. Vòng tròn xanh lá là mẫu vừa bị phân loại sai và được dùng để cập nhật; vùng xanh là vùng dự đoán đậu, vùng đỏ là vùng dự đoán rớt. Sau 5 *lần cập nhật, đường biên phân loại đúng cả* 6 *sinh viên.*

Nhận xét

- **Không phải lần cập nhật nào cũng sửa được mọi lỗi.** Mẫu 4 được sửa ở lần 2, nhưng đến lần cập nhật 3 (chỉnh theo mẫu 2) thì mẫu 4 lại bị sai trở lại, nên phải cập nhật một lần nữa. Sự dịch chuyển của đường biên là một quá trình "giằng co" giữa các mẫu, và thuật toán chỉ dừng khi không còn mẫu nào bị sai.
- **Kết quả khác với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ ở mục 2**, nhưng cũng phân loại đúng cả 6 mẫu. Điều này khớp với nhận xét ở mục 1.3: có vô số bộ tham số đúng, và thuật toán chỉ tìm ra **một** trong số đó, tùy thuộc vào khởi tạo và thứ tự duyệt mẫu.
- Perceptron **chỉ dừng khi đúng hết**, không cố gắng tìm đường biên "tốt nhất" theo nghĩa nào đó. Đường biên cuối cùng có thể rất sát một số mẫu.

3.6. Vai trò của tốc độ học

Ở hồi quy tuyến tính, tốc độ học η rất quan trọng: nếu chọn quá lớn, thuật toán có thể dao động hoặc phân kỳ, còn nếu quá nhỏ, thuật toán hội tụ rất chậm. Ta xem η ảnh hưởng thế nào đến Perceptron.

Khi khởi tạo $\mathbf{w} = \mathbf{0}$: η không làm đổi bộ phân loại

Ta chạy lại thuật toán trên dữ liệu sinh viên với ba giá trị $\eta = 0{,}1;\ 1;\ 10$, cùng khởi tạo $\mathbf{w} = \mathbf{0}$ và cùng thứ tự duyệt mẫu:

η	Số lần cập nhật	$\mathbf{w}$ cuối cùng
0,1	5	$[-0{,}1 \quad 0{,}4 \quad -0{,}8]^T$
1	5	$[-1 \quad 4 \quad -8]^T$
10	5	$[-10 \quad 40 \quad -80]^T$

Cả ba lần chạy đều **cập nhật đúng các mẫu giống nhau theo cùng thứ tự**, và $\mathbf{w}$ cuối cùng chỉ khác nhau một **hệ số tỉ lệ**: $\mathbf{w}^{(\eta)} = \eta\, \mathbf{w}^{(1)}$. Vì thế, ba bộ tham số này cho cùng một đường biên và cùng nhãn dự đoán.

Có thể giải thích điều này như sau. Từ mục 2.3, ta biết nhân $\mathbf{w}$ với một hằng số dương không đổi dấu của điểm số s_i, nên điều kiện cập nhật $m_i \le 0$ không đổi. Giả sử sau t bước, $\mathbf{w}_t^{(\eta)} = \eta\, \mathbf{w}_t^{(1)}$. Khi đó mẫu cần cập nhật ở bước tiếp theo trong hai lần chạy là như nhau, và

$$\mathbf{w}_{t+1}^{(\eta)} = \eta\, \mathbf{w}_t^{(1)} + \eta\, y_i\, \mathbf{x}_i^T = \eta\left(\mathbf{w}_t^{(1)} + y_i\, \mathbf{x}_i^T\right) = \eta\, \mathbf{w}_{t+1}^{(1)}$$

Vì $\mathbf{w}_0^{(\eta)} = \mathbf{0} = \eta \cdot \mathbf{0}$ nên theo quy nạp, tính chất này đúng ở mọi bước.

Khi khởi tạo $\mathbf{w}$ khác $\mathbf{0}$: η có ảnh hưởng

Nếu khởi tạo $\mathbf{w}^{(0)} \neq \mathbf{0}$, η thể hiện **tỉ lệ giữa bước cập nhật và độ lớn của $\mathbf{w}^{(0)}$**, nên ảnh hưởng đến quá trình huấn luyện. Ví dụ, với $\mathbf{w}^{(0)} = [1 \quad 1 \quad 1]^T$:

η	Số lần cập nhật	$\mathbf{w}$ cuối cùng
0,1	6	$[0{,}6 \quad 0{,}2 \quad -0{,}8]^T$
1	3	$[0 \quad 3 \quad -6]^T$
10	5	$[-9 \quad 41 \quad -79]^T$

Số lần cập nhật và bộ tham số cuối cùng đều khác nhau. Dù vậy, cả ba đều dẫn đến cùng một kết quả cuối: phân loại đúng toàn bộ dữ liệu.

Kết luận

- Với Perceptron, **hội tụ không phụ thuộc nhạy vào** η như ở hồi quy: miễn là dữ liệu phân tách tuyến tính được, thuật toán vẫn dừng sau hữu hạn bước với mọi $\eta > 0$.
- Khi khởi tạo $\mathbf{w} = \mathbf{0}$, η chỉ **co giãn** $\mathbf{w}$ mà không đổi bộ phân loại. Trong thực hành, ta thường chọn $\eta = 1$.
- Tình huống này **chỉ đúng với Perceptron**. Ở các mô hình có hàm mất mát trơn như hồi quy tuyến tính và Logistic Regression, η lại trở nên rất quan trọng, vì độ lớn của gradient phụ thuộc vào giá trị hiện tại của mô hình.

Tóm tắt mục 3

- Huấn luyện Perceptron là tìm $\mathbf{w}$ sao cho mọi margin $m_i = y_i\,\mathbf{x}_i\mathbf{w}$ đều dương. Số lỗi 0–1 không dùng được với Gradient Descent nên ta cần một hàm mất mát khác.
- **Hàm mất mát Perceptron** của một mẫu là $\ell_i = \max(0, -y_i\mathbf{x}_i\mathbf{w})$: bằng 0 nếu mẫu đúng, và bằng độ lớn của margin âm nếu mẫu sai. Hàm mất mát tổng là $J(\mathbf{w}) = \sum_i \ell_i$.
- Gradient của mất mát một mẫu sai là $-y_i\mathbf{x}_i^T$, dẫn đến **quy tắc cập nhật**: nếu $y_i\mathbf{x}_i\mathbf{w} \le 0$ thì $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T$. Nếu $y_i = +1$ thì **cộng** $\mathbf{x}_i^T$ vào $\mathbf{w}$, nếu $y_i = -1$ thì **trừ**. Mỗi lần cập nhật làm margin của mẫu đó tăng thêm $\eta \parallel \mathbf{x}_i \parallel^2$.
- Thuật toán duyệt từng mẫu, cập nhật ngay khi gặp mẫu cần cập nhật, lặp qua nhiều **epoch**, và dừng khi một epoch không có cập nhật nào hoặc khi đạt $T_{\max}$.
- Với khởi tạo $\mathbf{w} = \mathbf{0}$, tốc độ học η chỉ làm $\mathbf{w}$ tỉ lệ theo và **không đổi bộ phân loại**.

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -0{,}5]^T$, hãy tính $\mathbf{s}$, $\mathbf{m}$ và $J(\mathbf{w})$ trên 6 sinh viên của ví dụ.
2. Cho $\mathbf{w} = [0 \quad 1 \quad -1]^T$, $\eta = 1$ và một mẫu $\mathbf{x} = [1 \quad 2 \quad 3]$ có nhãn $y = +1$. Mẫu này có cần cập nhật không? Nếu có, hãy tính $\mathbf{w}$ mới và kiểm tra rằng margin tăng thêm đúng $\eta \parallel \mathbf{x} \parallel^2$.
3. Vì sao thuật toán dùng điều kiện cập nhật $m_i \le 0$ thay vì $m_i < 0$? Điều gì sẽ xảy ra nếu khởi tạo $\mathbf{w} = \mathbf{0}$ và dùng $m_i < 0$?
4. Giả sử khởi tạo $\mathbf{w} = \mathbf{0}$ và nhân đôi tốc độ học ($\eta \to 2\eta$). Vector $\mathbf{w}$ cuối cùng và nhãn dự đoán thay đổi thế nào?
5. Với $\mathbf{w} = [-1 \quad 4 \quad -8]^T$ (kết quả huấn luyện ở mục 3.5), hãy dự đoán nhãn của một sinh viên mới học 5 giờ và vắng 1,5 buổi.

Gợi ý đáp án

1. $\mathbf{s} = [-2{,}5 + 2 - 2 \quad -2{,}5 + 3 - 1{,}5 \quad -2{,}5 + 4 - 2{,}5 \quad -2{,}5 + 6 - 0{,}5 \quad -2{,}5 + 8 - 1 \quad -2{,}5 + 9]^T = [-2{,}5 \quad -1 \quad -1 \quad 3 \quad 4{,}5 \quad 6{,}5]^T$. Nhân với $\mathbf{y} = [-1 \quad -1 \quad -1 \quad +1 \quad +1 \quad +1]^T$ từng phần tử, ta được $\mathbf{m} = [2{,}5 \quad 1 \quad 1 \quad 3 \quad 4{,}5 \quad 6{,}5]^T$. Mọi margin đều dương nên $J(\mathbf{w}) = 0$.
2. $s = 0 + 2 - 3 = -1$ nên $m = (+1)(-1) = -1 \le 0$, cần cập nhật. $\mathbf{w} \leftarrow \begin{bmatrix}0\\1\\-1\end{bmatrix} + \begin{bmatrix}1\\2\\3\end{bmatrix} = \begin{bmatrix}1\\3\\2\end{bmatrix}$. Điểm số mới $s' = 1 + 6 + 6 = 13$ nên $m' = 13$. Margin tăng $13 - (-1) = 14 = \parallel \mathbf{x} \parallel^2 = 1 + 4 + 9$, đúng như công thức.
3. Với $\mathbf{w} = \mathbf{0}$, mọi mẫu có $s_i = 0$ và $m_i = 0$. Nếu chỉ cập nhật khi $m_i < 0$ thì **không mẫu nào được cập nhật**, epoch đầu tiên có $c = 0$ và thuật toán dừng ngay tại nghiệm vô nghĩa $\mathbf{w} = \mathbf{0}$. Điều kiện $m_i \le 0$ khắc phục điều này.
4. $\mathbf{w}$ cuối cùng nhân đôi, nhưng đường biên và nhãn dự đoán không đổi, vì các mẫu được cập nhật giống hệt như trước ($\mathbf{w}^{(2\eta)} = 2\,\mathbf{w}^{(\eta)}$).
5. $\mathbf{x} = [1 \quad 5 \quad 1{,}5]$ nên $s = -1 + 20 - 12 = 7 \ge 0$ và $\hat{y} = +1$ (dự đoán đậu).

4. Phân tích cơ chế học của Perceptron

Ở mục 3, ta đã xây dựng thuật toán Perceptron và chạy thử trên dữ liệu 6 sinh viên. Thuật toán dừng sau 5 lần cập nhật với $\mathbf{w} = [-1 \quad 4 \quad -8]^T$. Đến đây có ba câu hỏi tự nhiên cần được trả lời, nếu ta muốn hiểu thật sự Perceptron **học như thế nào** chứ không chỉ biết cách chạy nó:

- Kết quả huấn luyện có **duy nhất** không? Thứ tự duyệt mẫu và giá trị khởi tạo ảnh hưởng ra sao?
- Khi nào ta **chắc chắn** thuật toán sẽ dừng? Có thể ước lượng được số lần cập nhật không?
- Điều gì xảy ra khi **không thể** phân loại đúng hết bằng một đường thẳng?

Ta tiếp tục dùng quy ước ký hiệu cũ: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng** (phần tử đầu tiên bằng 1), vector tham số $\mathbf{w}$ là **vector cột**, nhãn $y_i \in \{-1, +1\}$, margin $m_i = y_i\, \mathbf{x}_i \mathbf{w}$. Thuật toán vẫn là thuật toán ở mục 3.4: cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T$ khi $m_i \le 0$.

4.1. Ảnh hưởng của thứ tự mẫu và khởi tạo tham số

Thứ tự duyệt mẫu

Vì Perceptron cập nhật $\mathbf{w}$ **ngay sau mỗi mẫu**, thứ tự duyệt mẫu quyết định mẫu nào được xét khi $\mathbf{w}$ đang có giá trị nào. Thứ tự khác nhau có thể dẫn đến các lần cập nhật khác nhau, và do đó đến kết quả khác nhau.

Ta thử nghiệm với dữ liệu 6 sinh viên, khởi tạo $\mathbf{w} = \mathbf{0}$, $\eta = 1$, và chạy thuật toán với **tất cả** $6! = 720$ **thứ tự duyệt mẫu** có thể có. Kết quả:

- Thuật toán **luôn dừng** và **luôn phân loại đúng cả** 6 **mẫu**, bất kể thứ tự.
- Số lần cập nhật dao động từ 2 đến 5: có 80 thứ tự cần 2 lần, 244 thứ tự cần 3 lần, 40 thứ tự cần 4 lần và 356 thứ tự cần 5 lần.
- Có tới 20 **bộ tham số cuối cùng khác nhau**.

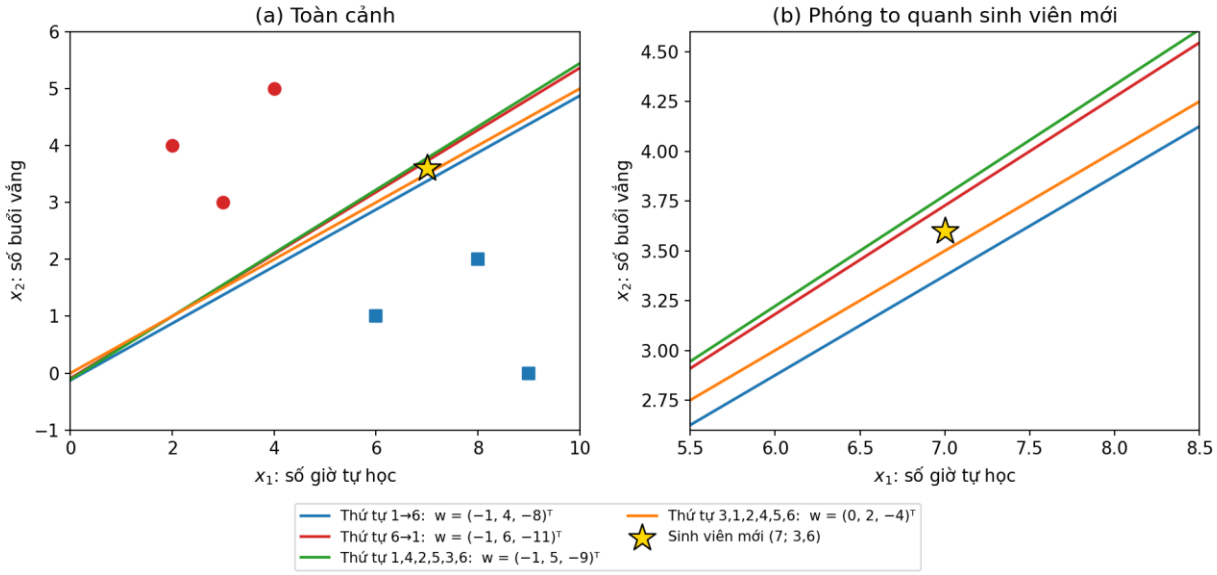
Bảng sau liệt kê bốn thứ tự cụ thể:

Thứ tự duyệt mẫu	Số lần cập nhật	$\mathbf{w}$ cuối cùng
1,2,3,4,5,6	5	$[-1 \quad 4 \quad -8]^T$
6,5,4,3,2,1	5	$[-1 \quad 6 \quad -11]^T$
1,4,2,5,3,6	5	$[-1 \quad 5 \quad -9]^T$
3,1,2,4,5,6	2	$[0 \quad 2 \quad -4]^T$

Cả bốn bộ tham số đều phân loại đúng 6 sinh viên huấn luyện, nhưng chúng là **bốn đường biên khác nhau** (Hình 8a). Sự khác biệt thể hiện rõ khi ta đưa vào một sinh viên **chưa từng thấy**: học 7 giờ, vắng 3,6 buổi, tức $\mathbf{x} = [1 \quad 7 \quad 3{,}6]$ (Hình 8b).

$\mathbf{w}$	Điểm số $s = \mathbf{x}\mathbf{w}$	Dự đoán
$[-1 \quad 4 \quad -8]^T$	$-1 + 28 - 28{,}8 = -1{,}8$	-1 (rớt)

$\mathbf{w}$	Điểm số $s = \mathbf{xw}$	Dự đoán
$[-1 \quad 6 \quad -11]^T$	$-1 + 42 - 39{,}6 = 1{,}4$	$+1$ (đậu)
$[-1 \quad 5 \quad -9]^T$	$-1 + 35 - 32{,}4 = 1{,}6$	$+1$ (đậu)
$[0 \quad 2 \quad -4]^T$	$14 - 14{,}4 = -0{,}4$	-1 (rớt)



Hình 8. Các đường biên do Perceptron tìm được với bốn thứ tự duyệt mẫu khác nhau. Cả bốn đều phân loại đúng 6 sinh viên huấn luyện, nhưng cho dự đoán khác nhau với sinh viên mới (ngôi sao): (a) toàn cảnh, (b) phóng to quanh sinh viên mới.

Nhận xét. Perceptron **không có nghiệm duy nhất**. Thuật toán chỉ đảm bảo dừng ở một nghiệm "đúng hết trên dữ liệu huấn luyện", không đảm bảo đó là nghiệm tốt nhất cho dữ liệu mới. Trong thực hành, người ta thường **xáo trộn ngẫu nhiên thứ tự mẫu ở đầu mỗi epoch** để kết quả không phụ thuộc vào cách dữ liệu được sắp xếp trong file.

Giá trị khởi tạo

Khởi tạo $\mathbf{w}^{(0)}$ cũng làm thay đổi quá trình học. Ta giữ nguyên $\eta = 1$ và thứ tự $1 \to 6$, chỉ đổi khởi tạo:

Khởi tạo $\mathbf{w}^{(0)}$	Số lần cập nhật	$\mathbf{w}$ cuối cùng
$[0 \quad 0 \quad 0]^T$	5	$[-1 \quad 4 \quad -8]^T$
$[-2 \quad 1 \quad 0]^T$	3	$[-3 \quad 2 \quad -6]^T$
$[1 \quad 1 \quad 1]^T$	3	$[0 \quad 3 \quad -6]^T$
$[0 \quad -1 \quad 0]^T$	5	$[-1 \quad 3 \quad -8]^T$
$[-2{,}5 \quad 1 \quad -1]^T$	0	$[-2{,}5 \quad 1 \quad -1]^T$

Dòng cuối cho thấy một điều dễ hiểu: nếu khởi tạo **đã** phân loại đúng toàn bộ dữ liệu thì thuật toán không có lần cập nhật nào và dừng ngay. Nói chung, **khởi tạo càng gần một nghiệm đúng thì càng cần ít lần cập nhật**.

Với Perceptron, ta thường khởi tạo $\mathbf{w} = \mathbf{0}$ vì cách này đơn giản và, như đã thấy ở mục 3.6, khi đó tốc độ học η không làm đổi bộ phân loại. Khác với mạng nơ-ron nhiều lớp (cần khởi tạo ngẫu nhiên để các nơ-ron không giống hệt nhau), một Perceptron đơn không gặp vấn đề này.

Kết luận

- Mọi thứ tự duyệt và mọi khởi tạo đều dẫn đến một bộ phân loại đúng toàn bộ dữ liệu huấn luyện (khi dữ liệu phân tách tuyến tính được, như ta sẽ giải thích ở mục 4.2).
- Nhưng **nghiệm cụ thể phụ thuộc vào thứ tự và khởi tạo**, và các nghiệm khác nhau có thể dự đoán khác nhau trên dữ liệu mới. Perceptron không có cơ chế nào để chọn trong số các nghiệm đúng ra một nghiệm "tốt hơn".
- Mong muốn chọn được đường biên **tốt nhất** theo một tiêu chí rõ ràng (ví dụ đường biên cách đều hai lớp nhất) là động lực để phát triển các mô hình khác, như Support Vector Machine.

4.2. Điều kiện hội tụ: dữ liệu phân tách tuyến tính

Câu hỏi: thuật toán có chắc chắn dừng không?

Ở mục 3.4, ta nói thuật toán dừng khi một epoch không có lần cập nhật nào. Nhưng điều gì đảm bảo ngày đó sẽ đến? Mỗi lần cập nhật cải thiện mẫu đang xét nhưng có thể làm xấu các mẫu khác, nên về nguyên tắc thuật toán có thể "giằng co" mãi. Thật may, có một kết quả toán học đẹp và rất cụ thể.

Định lý hội tụ của Perceptron

Ký hiệu R là độ dài lớn nhất của các vector đặc trưng (đã gồm phần tử 1 ở đầu):

$$R = \max_i \| \mathbf{x}_i \|$$

> **Định lý (Novikoff, 1962).** Giả sử tồn tại một vector $\mathbf{w}^*$ có độ dài $\| \mathbf{w}^* \| = 1$ và một số $\gamma > 0$ sao cho
>
> $$y_i \, \mathbf{x}_i \mathbf{w}^* \geq \gamma \qquad \text{với mọi } i = 1, \dots, n$$
>
> Khi đó, thuật toán Perceptron khởi tạo từ $\mathbf{w} = \mathbf{0}$, với mọi $\eta > 0$, thực hiện **không quá** R^2/γ^2 lần cập nhật:
>
> $$k \leq \frac{R^2}{\gamma^2}$$

Điều kiện của định lý chính là **dữ liệu phân tách tuyến tính được** (mục 1.3), nhưng được phát biểu mạnh hơn một chút: mọi mẫu không chỉ nằm đúng phía mà còn cách đường biên một khoảng tối thiểu theo thước đo γ. Số γ được gọi là **margin** của bộ phân loại $\mathbf{w}^*$ trên dữ liệu.

> **Lưu ý.** Độ dài $\|\cdot\|$ ở đây là độ dài Euclid của vector đầy đủ, **tính cả hệ số chệch** w_0 và phần tử 1 của $\mathbf{x}_i$. Vì vậy γ gần với khoảng cách hình học từ mẫu đến đường biên nhưng không trùng hoàn toàn.

Ví dụ với dữ liệu sinh viên. Chọn $\mathbf{w}^* = \frac{1}{\sqrt{21}}[-1 \quad 2 \quad -4]^T$ (có $\| \mathbf{w}^* \| = \sqrt{1+4+16}/\sqrt{21} = 1$). Với $\mathbf{w} = [-1 \quad 2 \quad -4]^T$, các margin m_i của 6 sinh viên lần lượt là 13,7,13,7,7,17, nên

$$\gamma = \min_i \frac{m_i}{\sqrt{21}} = \frac{7}{\sqrt{21}} \approx 1{,}53$$

Độ dài lớn nhất của $\mathbf{x}_i$ đạt tại sinh viên 6: $\| \mathbf{x}_6 \|^2 = 1^2 + 9^2 + 0^2 = 82$, nên $R^2 = 82$. Do đó

$$\frac{R^2}{\gamma^2} = \frac{82 \cdot 21}{49} \approx 35{,}1$$

Định lý cam kết thuật toán thực hiện **không quá** 35 **lần cập nhật**, bất kể thứ tự duyệt mẫu. Thực tế ở mục 3.5 chỉ cần 5 lần, và với 720 thứ tự ở mục 4.1, số lần lớn nhất cũng là 5. Như vậy cận trên của định lý là cận **an toàn nhưng khá rộng rãi**.

Chứng minh (phác thảo)

Ý tưởng chứng minh rất gọn: so sánh **hai đại lượng** cùng tăng theo số lần cập nhật nhưng tăng với tốc độ khác nhau.

Gọi $\mathbf{w}_k$ là vector tham số sau k lần cập nhật, với $\mathbf{w}_0 = \mathbf{0}$. Giả sử lần cập nhật thứ $k+1$ dùng mẫu i, tức là $y_i \, \mathbf{x}_i \mathbf{w}_k \le 0$ và $\mathbf{w}_{k+1} = \mathbf{w}_k + \eta \, y_i \, \mathbf{x}_i^T$.

Bước 1: $\mathbf{w}_k$ ngày càng "thẳng hàng" với $\mathbf{w}^*$. Nhân hai vế của quy tắc cập nhật với $(\mathbf{w}^*)^T$:

$$(\mathbf{w}^*)^T \mathbf{w}_{k+1} = (\mathbf{w}^*)^T \mathbf{w}_k + \eta \, y_i \, \mathbf{x}_i \mathbf{w}^* \ge (\mathbf{w}^*)^T \mathbf{w}_k + \eta\gamma$$

Mỗi lần cập nhật làm tích vô hướng này tăng ít nhất $\eta\gamma$. Vì $\mathbf{w}_0 = \mathbf{0}$, sau k lần cập nhật ta có

$$(\mathbf{w}^*)^T \mathbf{w}_k \ge k \, \eta \, \gamma$$

Bước 2: độ dài của $\mathbf{w}_k$ tăng chậm. Tính bình phương độ dài:

$$\| \mathbf{w}_{k+1} \|^2 = \| \mathbf{w}_k \|^2 + 2\eta \, y_i \, \mathbf{x}_i \mathbf{w}_k + \eta^2 \, \| \mathbf{x}_i \|^2 \le \| \mathbf{w}_k \|^2 + \eta^2 R^2$$

Ta đã dùng $y_i \, \mathbf{x}_i \mathbf{w}_k \le 0$ (đúng vì đây là mẫu được cập nhật) và $y_i^2 = 1$. Sau k lần cập nhật,

$$\| \mathbf{w}_k \|^2 \le k \, \eta^2 R^2$$

Bước 3: kết hợp. Theo bất đẳng thức Cauchy–Schwarz và $\| \mathbf{w}^* \| = 1$:

$$k \, \eta \, \gamma \le (\mathbf{w}^*)^T \mathbf{w}_k \le \| \mathbf{w}^* \| \, \| \mathbf{w}_k \| \le \eta R \sqrt{k}$$

Chia hai vế cho $\eta \sqrt{k} \, \gamma$ ta được $\sqrt{k} \le R/\gamma$, tức là $k \le R^2/\gamma^2$. ■

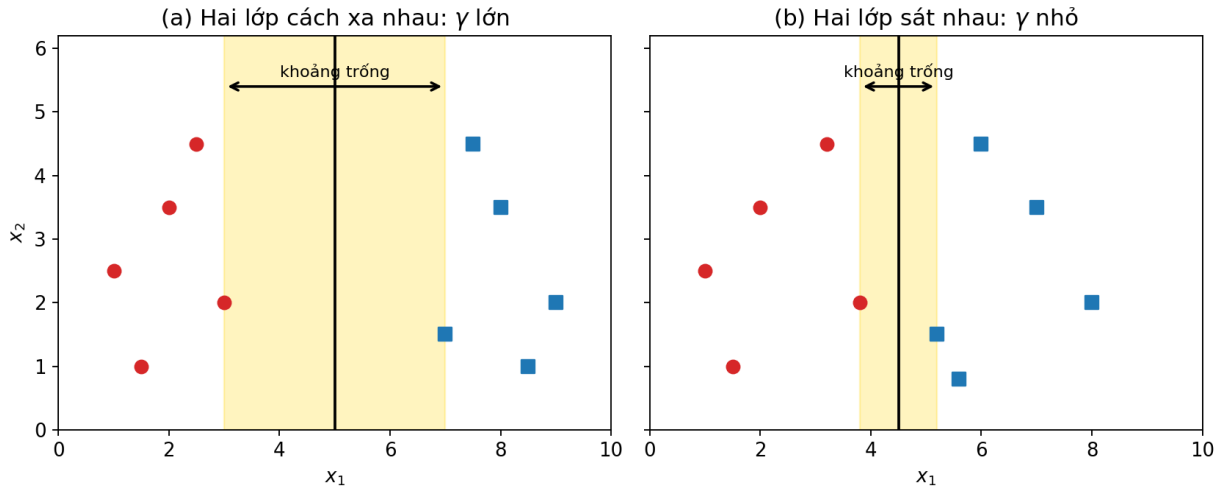
Diễn giải bằng lời. Sau k lần cập nhật, hình chiếu của $\mathbf{w}_k$ lên hướng $\mathbf{w}^*$ tăng **tuyến tính** theo k, nhưng độ dài của $\mathbf{w}_k$ chỉ tăng cỡ $\sqrt{k}$. Hình chiếu của một vector không thể lớn hơn độ dài của chính nó, nên k không thể tăng mãi: đến một lúc nào đó hai đại lượng này sẽ "va nhau", và thuật toán buộc phải dừng.

Ý nghĩa của định lý

Cận trên phụ thuộc R/γ, không phụ thuộc số mẫu n. Điều quan trọng với số lần cập nhật không phải là có bao nhiêu mẫu, mà là hai lớp **cách xa nhau đến mức nào** (so với kích thước dữ liệu):

- γ **lớn** (hai lớp cách xa nhau, có "khoảng trống" rộng giữa chúng): ít lần cập nhật.
- γ **nhỏ** (hai lớp sát nhau): nhiều lần cập nhật, vì đường biên phải điều chỉnh rất tinh.

Hình 9 minh họa ý này.



Hình 9. Hai lớp cách xa nhau (a) cho margin γ lớn, còn hai lớp sát nhau (b) cho margin γ nhỏ. Vùng vàng là "khoảng trống" giữa hai lớp.

Ta kiểm chứng bằng mô phỏng: tạo dữ liệu gồm hai lớp, mỗi lớp 50 điểm phân bố đều trong một hình vuông 4×4, hai hình vuông cách nhau một khoảng trống g theo trục x_1. Chạy Perceptron ($\mathbf{w}^{(0)} = \mathbf{0}$, $\eta = 1$) trên 30 bộ dữ liệu ngẫu nhiên cho mỗi giá trị g và lấy trung bình số lần cập nhật:

Khoảng trống g	0,25	0,5	1	2	4	8
Số lần cập nhật trung bình	9,7	5,6	3,4	2,0	1,5	1,0

Khoảng trống càng hẹp thì thuật toán càng phải cập nhật nhiều lần, đúng như định lý dự đoán.

Điều kiện cần và đủ cho việc dừng

Định lý cho ta chiều thuận: **nếu dữ liệu phân tách tuyến tính được thì thuật toán dừng sau hữu hạn bước**. Chiều ngược lại hiển nhiên đúng: nếu thuật toán dừng thì ở epoch cuối không có mẫu nào có $m_i \le 0$, tức là $y_i \, \mathbf{x}_i \mathbf{w} > 0$ với mọi i, và chính $\mathbf{w}$ đó chứng tỏ dữ liệu phân tách được. Tóm lại:

$$\text{Dữ liệu phân tách tuyến tính được} \iff \text{Perceptron dừng sau hữu hạn lần cập nhật}$$

Hội tụ không có nghĩa là tốt nhất

Cần tránh hiểu lầm: định lý chỉ nói rằng Perceptron **tìm được một đường biên đúng toàn bộ dữ liệu huấn luyện**. Thuật toán dừng ngay khi hết mẫu sai, nên đường biên có thể rất sát một số mẫu (margin nhỏ), như ta đã thấy ở mục 4.1. Một đường biên sát mẫu thường kém ổn định trước nhiễu và dữ liệu mới. Ý tưởng **tìm đường biên có margin lớn nhất** dẫn đến Support Vector Machine, nằm ngoài phạm vi bài này.

4.3. Trường hợp không hội tụ: bài toán XOR

Bài toán XOR

XOR (hoặc loại trừ) là phép toán logic cho kết quả "đúng" khi hai đầu vào **khác nhau**. Với nhãn $\{-1, +1\}$, ta gán $y = +1$ nếu hai đầu vào khác nhau và $y = -1$ nếu giống nhau:

x_1	x_2	y
0	0	-1
0	1	$+1$
1	0	$+1$
1	1	-1

Ma trận đặc trưng và vector nhãn là

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -1 \\ +1 \\ +1 \\ -1 \end{bmatrix}$$

Trên mặt phẳng, bốn điểm này nằm ở bốn đỉnh của hình vuông, và hai điểm cùng lớp nằm ở **hai đỉnh đối diện** (Hình 10a). Nhìn vào hình, ta thấy không có đường thẳng nào tách được hai lớp.

Chứng minh XOR không phân tách tuyến tính được

Dữ liệu phân tách tuyến tính được khi tồn tại $\mathbf{w} = [w_0 \quad w_1 \quad w_2]^T$ sao cho $y_i \, \mathbf{x}_i \mathbf{w} > 0$ với cả bốn mẫu. Viết ra từng mẫu:

$$\begin{cases} (-1)(w_0) > 0 & \Leftrightarrow \quad w_0 < 0 & \text{(mẫu 1)} \\ (+1)(w_0 + w_2) > 0 & \Leftrightarrow \quad w_0 + w_2 > 0 & \text{(mẫu 2)} \\ (+1)(w_0 + w_1) > 0 & \Leftrightarrow \quad w_0 + w_1 > 0 & \text{(mẫu 3)} \\ (-1)(w_0 + w_1 + w_2) > 0 & \Leftrightarrow \quad w_0 + w_1 + w_2 < 0 & \text{(mẫu 4)} \end{cases}$$

Cộng vế với vế của mẫu 2 và mẫu 3:

$$2w_0 + w_1 + w_2 > 0$$

Cộng vế với vế của mẫu 1 và mẫu 4:

$$2w_0 + w_1 + w_2 < 0$$

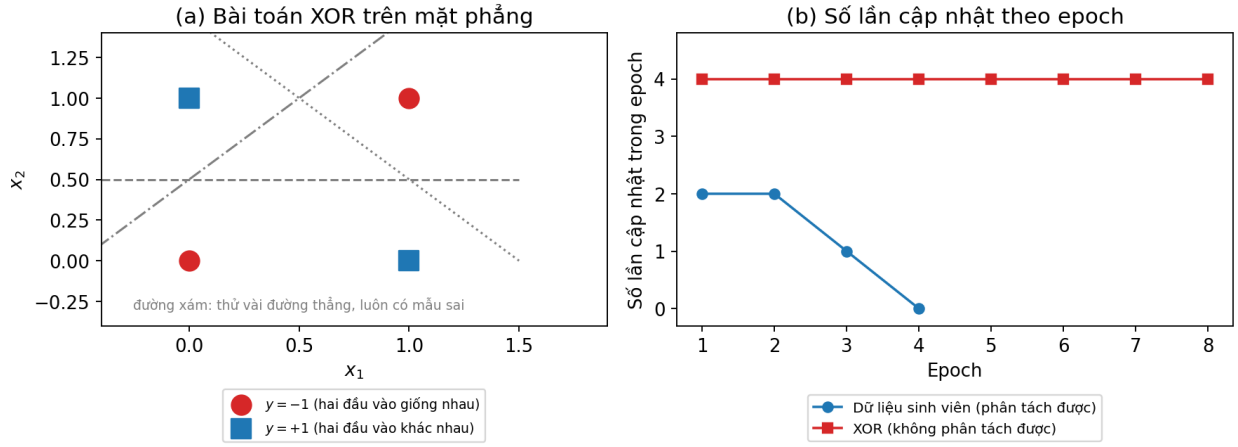
Hai kết quả mâu thuẫn nhau. Vậy **không tồn tại w** nào thỏa cả bốn điều kiện, tức là XOR không phân tách tuyến tính được. Kết luận này đúng với mọi mô hình có đường biên là siêu phẳng, không riêng Perceptron.

Perceptron làm gì với XOR?

Ta chạy thuật toán với $\mathbf{w}^{(0)} = \mathbf{0}$, $\eta = 1$, thứ tự $1 \to 4$.

- **Mẫu 1** ($\mathbf{x}_1 = [1 \quad 0 \quad 0]$, $y_1 = -1$): $s = 0$, $m = 0 \le 0$. Cập nhật: $\mathbf{w} \leftarrow \mathbf{0} - \mathbf{x}_1^T = [-1 \quad 0 \quad 0]^T$.
- **Mẫu 2** ($\mathbf{x}_2 = [1 \quad 0 \quad 1]$, $y_2 = +1$): $s = -1$, $m = -1 \le 0$. Cập nhật: $\mathbf{w} \leftarrow [-1 \quad 0 \quad 0]^T + \mathbf{x}_2^T = [0 \quad 0 \quad 1]^T$.
- **Mẫu 3** ($\mathbf{x}_3 = [1 \quad 1 \quad 0]$, $y_3 = +1$): $s = 0$, $m = 0 \le 0$. Cập nhật: $\mathbf{w} \leftarrow [0 \quad 0 \quad 1]^T + \mathbf{x}_3^T = [1 \quad 1 \quad 1]^T$.
- **Mẫu 4** ($\mathbf{x}_4 = [1 \quad 1 \quad 1]$, $y_4 = -1$): $s = 3$, $m = -3 \le 0$. Cập nhật: $\mathbf{w} \leftarrow [1 \quad 1 \quad 1]^T - \mathbf{x}_4^T = [0 \quad 0 \quad 0]^T$.

Sau epoch đầu tiên, **w quay về đúng 0**, tức là trạng thái ban đầu. Do đó epoch thứ hai lặp lại y hệt epoch đầu, và cứ thế mãi: **mỗi epoch đều có 4 lần cập nhật, và w quay vòng không bao giờ dừng**. Hình 10b so sánh với dữ liệu sinh viên, nơi số lần cập nhật giảm về 0 sau 4 epoch.



Hình 10. (a) Bài toán XOR: hai điểm cùng lớp nằm ở hai đỉnh đối diện nên không đường thẳng nào tách được. (b) Số lần cập nhật theo epoch: dữ liệu sinh viên giảm về 0, còn XOR luôn bằng 4.

Đây là đặc điểm chung của dữ liệu không phân tách được: luôn tồn tại ít nhất một mẫu có $m_i \le 0$ ở mọi epoch, nên điều kiện dừng $c = 0$ **không bao giờ xảy ra**. Vì vậy thuật toán ở mục 3.4 cần điều kiện dừng thứ hai là **số epoch tối đa** $T_{\max}$.

Tốt nhất có thể là bao nhiêu?

Với XOR, một đường thẳng bất kỳ luôn sai ít nhất một trong bốn mẫu. Một ví dụ phân loại đúng 3 mẫu là $\mathbf{w} = [-0{,}5 \quad 1 \quad 1]^T$, tức $s = -0{,}5 + x_1 + x_2$: mẫu 1,2,3 đúng và mẫu 4 sai. Độ chính xác tối đa mà mô hình tuyến tính đạt được trên XOR là $3/4 = 75\%$.

Khi dữ liệu không phân tách được, thuật toán Perceptron kinh điển dao động giữa nhiều nghiệm và điểm dừng tại $T_{\max}$ có thể là một nghiệm kém. Một cách khắc phục đơn giản là **thuật toán túi (pocket algorithm)**:

1. Chạy Perceptron như bình thường, nhưng luôn **giữ lại trong "túi"** bộ tham số $\mathbf{w}_{\text{túi}}$ có **ít mẫu bị sai nhất** từ đầu đến giờ.
2. Mỗi khi $\mathbf{w}$ hiện tại phân loại tốt hơn $\mathbf{w}_{\text{túi}}$, thay $\mathbf{w}_{\text{túi}}$ bằng $\mathbf{w}$.
3. Khi dừng (đạt $T_{\max}$), trả về $\mathbf{w}_{\text{túi}}$ thay vì $\mathbf{w}$ cuối cùng.

Hướng khắc phục: thay đổi cách biểu diễn dữ liệu

Giới hạn trên là của **đường biên phẳng**, không phải của việc huấn luyện. Nếu ta **thay đổi đặc trưng** để dữ liệu trở nên phân tách được trong không gian mới, Perceptron lại hoạt động được. Với XOR, thêm một đặc trưng là tích x_1x_2. Vector đặc trưng mới và ma trận đặc trưng mới là

$$\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad x_1x_2], \qquad \mathbf{X}' = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Chọn $\mathbf{w}' = [-0{,}5 \quad 1 \quad 1 \quad -2]^T$, ta có

$$\mathbf{s} = \mathbf{X}'\mathbf{w}' = \begin{bmatrix} -0{,}5 \\ 0{,}5 \\ 0{,}5 \\ -0{,}5 \end{bmatrix} \quad \Rightarrow \quad \hat{\mathbf{y}} = \begin{bmatrix} -1 \\ +1 \\ +1 \\ -1 \end{bmatrix} = \mathbf{y}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ 0{,}5 \end{bmatrix}$$

Mọi margin đều dương, nên XOR đã được phân loại đúng hoàn toàn. Mô hình vẫn **tuyến tính theo đặc trưng mới $\mathbf{x}'$**, nhưng trong không gian đặc trưng gốc (x_1, x_2) thì đường biên là một đường **cong**. Tuy nhiên, cách này đòi hỏi ta phải tự đoán ra đặc trưng phù hợp.

Một hướng tổng quát hơn là **ghép nhiều Perceptron thành nhiều lớp**, để các lớp đầu tự học ra những đặc trưng hữu ích cho lớp sau. XOR có thể biểu diễn bằng cách kết hợp các phép logic mà Perceptron đơn làm được (chẳng hạn "x_1 HOẶC x_2" và "KHÔNG PHẢI (x_1 VÀ x_2)"). Năm 1969, Marvin Minsky và Seymour Papert đã chỉ ra giới hạn của Perceptron đơn trên các bài toán như XOR, góp phần làm giảm sự quan tâm đến mạng nơ-ron trong một thời gian, cho đến khi các mạng nhiều lớp được huấn luyện hiệu quả. Ta sẽ quay lại ý tưởng này ở mục 6.

> **Trong thực tế.** Dữ liệu thật thường có nhiễu, nên hiếm khi phân tách tuyến tính được một cách hoàn hảo. Perceptron thuần thường không dừng trên dữ liệu thật. Vì vậy ta cần $T_{\max}$, thuật toán túi, hoặc chuyển sang các mô hình có hàm mất mát trơn hơn như Logistic Regression.

Tóm tắt mục 4

- Kết quả huấn luyện Perceptron **không duy nhất**: thứ tự duyệt mẫu và giá trị khởi tạo cho ra các bộ tham số khác nhau, đều đúng trên dữ liệu huấn luyện nhưng có thể dự đoán khác nhau trên dữ liệu mới. Khởi tạo càng gần nghiệm đúng thì cần càng ít cập nhật.
- **Định lý hội tụ:** nếu dữ liệu phân tách tuyến tính được với margin γ và $\| \mathbf{x}_i \| \leq R$, thuật toán thực hiện không quá R^2/γ^2 lần cập nhật. Cận này không phụ thuộc số mẫu n.

- Chứng minh dựa trên hai bất đẳng thức: $(\mathbf{w}^*)^T\mathbf{w}_k \geq k\eta\gamma$ (tăng tuyến tính) và $\| \mathbf{w}_k \|^2 \leq k\eta^2R^2$ (độ dài tăng chậm).
- Dữ liệu phân tách tuyến tính được **khi và chỉ khi** Perceptron dừng sau hữu hạn bước. Hội tụ không có nghĩa là đường biên tốt nhất.
- **XOR** không phân tách tuyến tính được (có thể chứng minh bằng 4 bất đẳng thức mâu thuẫn). Perceptron khi đó **quay vòng mãi**, nên cần $T_{\max}$, và có thể dùng thuật toán túi để giữ nghiệm tốt nhất.
- Hạn chế của đường biên phẳng được khắc phục bằng cách **thêm đặc trưng phi tuyến** hoặc **ghép nhiều lớp Perceptron** (mạng nơ-ron).

Câu hỏi kiểm tra nhanh

1. Cho $\mathbf{w} = [0 \quad 1 \quad 0]^T$, $\eta = 1$ và mẫu $\mathbf{x} = [1 \quad 2 \quad 1]$ có nhãn $y = -1$. Mẫu có cần cập nhật không? Nếu có, hãy tính $\mathbf{w}$ mới và kiểm tra margin mới theo công thức $m' = m + \eta \| \mathbf{x} \|^2$.
2. Một bộ dữ liệu có $R = 10$ và margin $\gamma = 0{,}5$. Định lý hội tụ cho cận trên bao nhiêu lần cập nhật? Nếu hai lớp được tách xa gấp đôi (γ tăng gấp đôi, R giữ nguyên) thì cận trên thay đổi thế nào?
3. Kiểm tra rằng $\mathbf{w} = [-1{,}5 \quad 1 \quad 1]^T$ phân loại đúng cổng AND (nhãn $+1$ chỉ khi $x_1 = x_2 = 1$, còn lại -1), và tính vector margin $\mathbf{m}$.
4. Vì sao hai lần huấn luyện với thứ tự duyệt mẫu khác nhau có thể cho hai bộ tham số khác nhau, dù cả hai đều phân loại đúng 100% dữ liệu huấn luyện? Điều này dẫn đến hệ quả gì khi dự đoán dữ liệu mới?
5. Vì sao một bộ dữ liệu có nhãn nhiễu (một vài mẫu bị gán sai nhãn) thường làm Perceptron không dừng? Ta nên làm gì?
6. Với đặc trưng mở rộng $\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad x_1x_2]$ và $\mathbf{w}' = [-0{,}5 \quad 1 \quad 1 \quad -2]^T$, hãy tính điểm số và nhãn dự đoán cho mẫu $(x_1, x_2) = (1,0)$.

Gợi ý đáp án

1. $s = 0 + 2 + 0 = 2$ nên $m = (-1)(2) = -2 \leq 0$, cần cập nhật. $\mathbf{w} \leftarrow \begin{bmatrix}0\\1\\0\end{bmatrix} - \begin{bmatrix}1\\2\\1\end{bmatrix} = \begin{bmatrix}-1\\-1\\-1\end{bmatrix}$. Điểm số mới $s' = -1 - 2 - 1 = -4$ nên $m' = (-1)(-4) = 4$. Công thức: $m' = -2 + \| \mathbf{x} \|^2 = -2 + (1 + 4 + 1) = 4$, khớp.
2. Cận trên là $R^2/\gamma^2 = 100/0{,}25 = 400$ lần cập nhật. Khi γ tăng gấp đôi thì γ^2 tăng gấp bốn, nên cận trên giảm còn $1/4$, tức 100 lần.
3. Điểm số: $s = -1{,}5 + x_1 + x_2$ cho $\mathbf{s} = [-1{,}5 \quad -0{,}5 \quad -0{,}5 \quad 0{,}5]^T$ trên bốn mẫu $(0,0), (0,1), (1,0), (1,1)$. Nhãn thực $\mathbf{y} = [-1 \quad -1 \quad -1 \quad +1]^T$. Vector margin $\mathbf{m} = \mathbf{y} \odot \mathbf{s} = [1{,}5 \quad 0{,}5 \quad 0{,}5 \quad 0{,}5]^T$, tất cả đều dương nên phân loại đúng cả 4 mẫu.
4. Mỗi thứ tự khiến thuật toán gặp các mẫu sai ở những thời điểm khác nhau nên các lần cập nhật khác nhau. Có vô số đường biên đúng toàn bộ dữ liệu huấn luyện, và thuật toán dừng ở một đường bất kỳ trong số đó. Hệ quả là các đường biên này có thể dự đoán **khác nhau** với dữ liệu mới, và Perceptron không có cách nào để biết đường nào tốt hơn.
5. Mẫu bị gán sai nhãn có thể nằm lẫn vào vùng của lớp kia, làm dữ liệu **không còn phân tách tuyến tính được**. Khi đó luôn còn ít nhất một mẫu có $m_i \leq 0$ ở mọi epoch, nên điều

kiện dừng $c = 0$ không xảy ra. Ta cần đặt $T_{\max}$, có thể dùng thuật toán túi, hoặc dùng mô hình chịu được nhiễu tốt hơn như Logistic Regression.

6. $\mathbf{x}' = [1 \quad 1 \quad 0 \quad 0]$ nên $s = -0{,}5 + 1 + 0 - 0 = 0{,}5 \geq 0$ và $\hat{y} = +1$. Đúng với nhãn XOR của mẫu $(1{,}0)$.

5. Cài đặt và đánh giá mô hình Perceptron

Ở các mục trước, ta đã xây dựng Perceptron bằng toán học và tính tay trên 6 sinh viên. Mục này biến lý thuyết thành **chương trình chạy được** và trả lời một câu hỏi quan trọng không kém việc huấn luyện: **mô hình học được tốt đến đâu?** Nội dung gồm ba phần:

- Cài đặt thuật toán Perceptron **từ đầu** bằng Python (chỉ dùng NumPy) và kiểm tra rằng kết quả khớp với ví dụ tính tay ở mục 3.5.
- Huấn luyện trên một bộ dữ liệu lớn hơn, **chia dữ liệu thành tập huấn luyện và tập kiểm tra**, rồi đánh giá bằng accuracy, ma trận nhầm lẫn, precision, recall và F1.
- So sánh với lớp `Perceptron` của **scikit-learn**.

Toàn bộ mã trong mục này đã được chạy thật, và các kết quả in ra đều lấy từ lần chạy đó. Ta dùng ba thư viện: NumPy (tính toán ma trận), Matplotlib (vẽ hình) và scikit-learn (so sánh).

Quy ước chuyển từ ký hiệu toán sang mã

Ta giữ nguyên quy ước ký hiệu của toàn bài: **chữ hoa in đậm** là ma trận, **chữ thường in đậm** là vector, $\mathbf{x}_i$ là vector hàng, $\mathbf{w}$ là vector cột, nhãn $y_i \in \{-1, +1\}$. Trong NumPy, ma trận là mảng hai chiều, còn vector là mảng một chiều (NumPy tự hiểu là vector hàng hay cột tùy phép nhân). Bảng sau cho biết mỗi công thức ứng với dòng mã nào:

Ký hiệu toán	Trong mã Python
Ma trận đặc trưng $\mathbf{X}$ (kích thước $n \times (d+1)$, cột đầu bằng 1)	`Xb` (kết quả của hàm `them_bias`)
Vector tham số $\mathbf{w}$ (độ dài $d+1$)	`w`
Vector đặc trưng của mẫu i, $\mathbf{x}_i$	`Xb[i]`
Điểm số $s_i = \mathbf{x}_i \mathbf{w}$	`Xb[i] @ w`
Vector điểm số $\mathbf{s} = \mathbf{X}\mathbf{w}$	`Xb @ w`
Nhãn dự đoán $\hat{\mathbf{y}} = \text{sign}(\mathbf{s})$	`np.where(s >= 0, 1, -1)`
Margin $m_i = y_i\, s_i$	`y[i] * (Xb[i] @ w)`
Cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i^T$	`w = w + eta * y[i] * Xb[i]`

5.1. Cài đặt Perceptron từ đầu bằng Python

Thêm hệ số chệch vào ma trận đặc trưng

Dữ liệu thô là ma trận $n \times d$ chứa các đặc trưng. Ta cần thêm cột toàn số 1 ở đầu để được ma trận $\mathbf{X}$ kích thước $n \times (d+1)$, như đã làm ở mục 1:

```
import numpy as np

def them_bias(X):
```

```
    """Thêm cột 1 vào đầu ma trận đặc trưng: (n, d) -> (n, d+1)."""
    return np.hstack([np.ones((X.shape[0], 1)), X])
```

Hàm dự đoán

Hàm này thực hiện đúng quy tắc $\hat{\mathbf{y}} = \text{sign}(\mathbf{Xw})$ với quy ước $\text{sign}(0) = +1$ ở mục 2. Chú ý rằng toàn bộ tính toán là **một phép nhân ma trận** chứ không cần vòng lặp qua từng mẫu:

```
def du_doan(Xb, w):
    """Nhãn dự đoán y_hat = sign(Xw), với quy ước sign(0) = +1."""
    s = Xb @ w
    return np.where(s >= 0, 1, -1)
```

Hàm huấn luyện

Hàm sau cài đặt thuật toán ở mục 3.4. Có thể đối chiếu từng dòng với các bước của thuật toán: khởi tạo $\mathbf{w} = \mathbf{0}$, lặp theo epoch, duyệt từng mẫu, kiểm tra $m_i \le 0$, cập nhật ngay, và dừng khi một epoch không có cập nhật nào hoặc khi đạt $T_{\max}$:

```
def huan_luyen(Xb, y, eta=1.0, t_max=100):
    """Thuật toán Perceptron: cập nhật ngay sau mỗi mẫu khi y_i * (x_i w) <=
0."""
    n, d1 = Xb.shape
    w = np.zeros(d1)                        # khởi tạo w = 0
    lich_su = []                            # số lần cập nhật ở mỗi epoch
    quy_dao = []                            # w sau mỗi lần cập nhật
    for epoch in range(t_max):
        so_cap_nhat = 0
        for i in range(n):
            m = y[i] * (Xb[i] @ w)          # margin của mẫu i
            if m <= 0:
                w = w + eta * y[i] * Xb[i]
                quy_dao.append(w.copy())
                so_cap_nhat += 1
        lich_su.append(so_cap_nhat)
        if so_cap_nhat == 0:                # một epoch không cập nhật: hội tụ
            break
    return w, lich_su, quy_dao
```

Hàm trả về ba thứ: vector tham số cuối cùng *w*, số lần cập nhật ở từng epoch *lich_su* (để theo dõi quá trình học), và danh sách *quy_dao* ghi lại **w** sau mỗi lần cập nhật (để vẽ hình).

Kiểm tra trên dữ liệu 6 sinh viên

Cách tốt nhất để biết cài đặt đúng là so với kết quả đã tính tay. Ta chạy trên đúng 6 sinh viên của mục 3.5:

```
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0]], dtype=float
)
y_sv = np.array([-1, -1, -1, 1, 1, 1])
Xb_sv = them_bias(X_sv)
```

```
w, lich_su, quy_dao = huan_luyen(Xb_sv, y_sv)
print("w =", w)
print("So cap nhat moi epoch:", lich_su)
print("y_hat =", du_doan(Xb_sv, w))
print("margin m =", y_sv * (Xb_sv @ w))
print(np.array(quy_dao))
```

Kết quả in ra:

```
w = [-1.  4. -8.]
So cap nhat moi epoch: [2, 2, 1, 0]
y_hat = [-1 -1 -1  1  1  1]
margin m = [25. 13. 25. 15. 15. 35.]
[[-1. -2. -4.]
 [ 0.  4. -3.]
 [-1.  1. -6.]
 [ 0.  7. -5.]
 [-1.  4. -8.]]
```

Mọi thứ **khớp hoàn toàn với ví dụ tính tay**: $\mathbf{w} = [-1 \quad 4 \quad -8]^T$; số lần cập nhật ở bốn epoch lần lượt là 2,2,1,0; vector margin là $[25 \quad 13 \quad 25 \quad 15 \quad 15 \quad 35]^T$; và 5 dòng của *quy_dao* chính là 5 lần cập nhật trong bảng ở mục 3.5.

Trực quan hóa quá trình học

Danh sách *quy_dao* cho phép ta vẽ lại đường biên sau mỗi lần cập nhật, giống Hình 7 ở mục 3.5. Hàm sau vẽ các điểm dữ liệu cùng đường biên và hai vùng dự đoán cho một bộ **w** bất kỳ:

```
import matplotlib.pyplot as plt

def ve_duong_bien(ax, X, y, w, x1_max=10, x2_max=6):
    """Vẽ dữ liệu, đường biên w0 + w1*x1 + w2*x2 = 0 và hai vùng dự đoán."""
    u, v = np.meshgrid(np.linspace(0, x1_max, 200), np.linspace(-1, x2_max, 2
00))
    s = w[0] + w[1] * u + w[2] * v
    ax.contourf(u, v, np.where(s >= 0, 1, -1),
                levels=[-2, 0, 2], colors=["tab:red", "tab:blue"], alpha=0.08
)
    ax.contour(u, v, s, levels=[0], colors="k")           # đường biên s = 0
    ax.scatter(*X[y == -1].T, c="tab:red", label="Rớt")
    ax.scatter(*X[y == 1].T, c="tab:blue", marker="s", label="Đậu")
    ax.set_xlim(0, x1_max)
    ax.set_ylim(-1, x2_max)

fig, axes = plt.subplots(1, 5, figsize=(18, 3.6))
for ax, w_k in zip(axes, quy_dao):
    ve_duong_bien(ax, X_sv, y_sv, w_k)
```

Đoạn mã này cho năm biểu đồ, mỗi biểu đồ ứng với một lần cập nhật, và nội dung giống với Hình 7. Hàm *ve_duong_bien* cũng sẽ được dùng lại ở mục 5.2.

5.2. Huấn luyện trên dữ liệu lớn hơn và đánh giá mô hình

Với 6 mẫu, ta không thể nói gì về chất lượng mô hình trên dữ liệu mới. Mục này dùng một bộ dữ liệu lớn hơn và đánh giá một cách bài bản.

Dữ liệu

Ta **mô phỏng** 400 sinh viên, mỗi lớp 200 người, với hai đặc trưng như trước: x_1 là số giờ tự học và x_2 là số buổi vắng. Sinh viên đậu học nhiều và vắng ít hơn, sinh viên rớt thì ngược lại, nhưng hai nhóm **chồng lấn nhau** đáng kể, giống dữ liệu thật:

```
rng = np.random.default_rng(42)
n_lop = 200
dau = np.c_[rng.normal(8, 2.2, n_lop), rng.normal(1.5, 1.2, n_lop)]   # lớp +1 (đậu)
rot = np.c_[rng.normal(4, 2.2, n_lop), rng.normal(3.5, 1.5, n_lop)]   # lớp -1 (rớt)
X = np.clip(np.vstack([dau, rot]), 0, None)        # giờ học, buổi vắng không âm
y = np.r_[np.ones(n_lop), -np.ones(n_lop)].astype(int)
```

Tập huấn luyện và tập kiểm tra

Nếu ta dùng **toàn bộ** dữ liệu để huấn luyện rồi cũng dùng chính dữ liệu đó để đánh giá, ta chỉ đo được mô hình **nhớ** dữ liệu cũ tốt đến đâu, chứ không đo được nó **dự đoán** dữ liệu mới tốt đến đâu. Điều này giống như một học sinh ôn đúng đề rồi thi lại chính đề đó: điểm cao chưa chứng tỏ hiểu bài.

Vì vậy, ta **chia dữ liệu thành hai phần** ngay từ đầu:

- **Tập huấn luyện (training set):** dùng để học **w**. Ở đây là 70% dữ liệu.
- **Tập kiểm tra (test set):** được **giấu đi** trong suốt quá trình huấn luyện và chỉ dùng **một lần ở cuối** để đánh giá. Ở đây là 30% dữ liệu.

Độ chính xác trên tập kiểm tra là ước lượng hợp lý cho chất lượng của mô hình trên dữ liệu chưa từng thấy. Hàm sau xáo trộn dữ liệu rồi chia:

```
def chia_train_test(X, y, ti_le_test=0.3, seed=0):
    """Xáo trộn rồi chia dữ liệu thành tập huấn luyện và tập kiểm tra."""
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    n_test = int(len(y) * ti_le_test)
    return X[idx[n_test:]], X[idx[:n_test]], y[idx[n_test:]], y[idx[:n_test]]

X_tr, X_te, y_tr, y_te = chia_train_test(X, y, ti_le_test=0.3, seed=0)
Xb_tr, Xb_te = them_bias(X_tr), them_bias(X_te)
```

Ta được 280 mẫu huấn luyện (trong đó 149 sinh viên đậu) và 120 mẫu kiểm tra (trong đó 51 sinh viên đậu).

Quy tắc vàng. Tập kiểm tra không được dùng để chọn tham số, chọn $T_{\max}$, hay bất kỳ quyết định nào trong khi huấn luyện. Nếu vi phạm, ước lượng trên tập kiểm tra sẽ quá lạc quan.

Huấn luyện: dữ liệu chồng lấn nên không hội tụ

Ta chạy thuật toán Perceptron thuần túy với $T_{\max} = 100$:

```
w_1, lich_su_1, _ = huan_luyen(Xb_tr, y_tr, t_max=100)
print("so epoch da chay:", len(lich_su_1), "| so cap nhat 5 epoch cuoi:", lic
h_su_1[-5:])
```

```
so epoch da chay: 100 | so cap nhat 5 epoch cuoi: [42, 45, 44, 42, 49]
```

Thuật toán chạy đủ 100 epoch mà **mỗi epoch vẫn còn hàng chục lần cập nhật**. Đúng như phân tích ở mục 4.3: dữ liệu của hai lớp chồng lấn nên **không phân tách tuyến tính được**, và điều kiện dừng "không còn cập nhật" không bao giờ xảy ra. Đây là tình huống thường gặp với dữ liệu thật.

Thuật toán túi

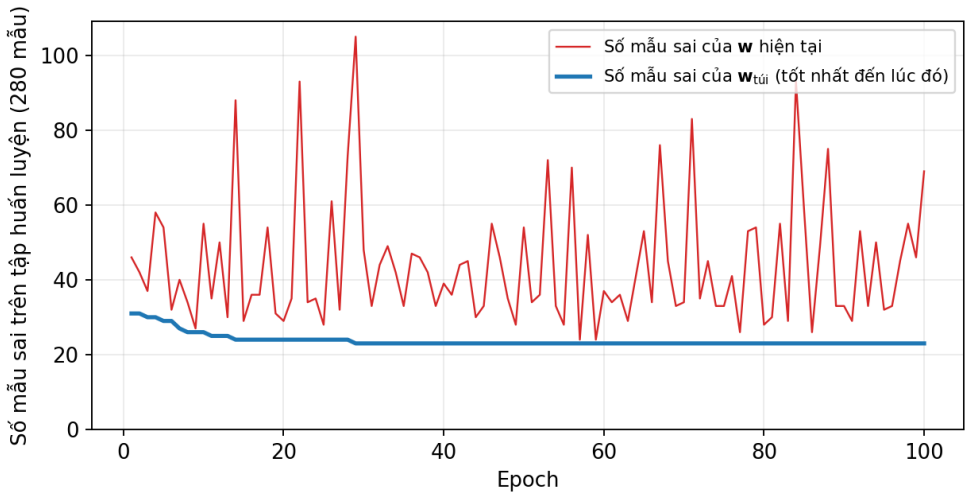
Khi không hội tụ, bộ **w** ở epoch cuối cùng có thể rất tệ vì nó dao động. Ta cài đặt **thuật toán túi** đã mô tả ở mục 4.3: sau mỗi lần cập nhật, kiểm tra số mẫu sai trên tập huấn luyện, và nếu tốt hơn mọi lần trước thì cất **w** đó vào "túi". Ta cũng **xáo trộn thứ tự mẫu ở mỗi epoch** như đã khuyến nghị ở mục 4.1:

```
def huan_luyen_tui(Xb, y, eta=1.0, t_max=100, seed=0):
    """Perceptron + thuật toán túi: giữ lại w có ít mẫu sai nhất."""
    n = Xb.shape[0]
    rng = np.random.default_rng(seed)
    w = np.zeros(Xb.shape[1])
    w_tui = w.copy()
    loi_tui = np.sum(du_doan(Xb, w) != y)
    lich_su = []                            # (số mẫu sai của w, số mẫu sai của
w_tui)
    for epoch in range(t_max):
        for i in rng.permutation(n):        # xáo trộn thứ tự mẫu ở mỗi epoch
            if y[i] * (Xb[i] @ w) <= 0:
                w = w + eta * y[i] * Xb[i]
                loi = np.sum(du_doan(Xb, w) != y)
                if loi < loi_tui:           # tốt hơn túi hiện tại: bỏ vào túi
                    w_tui, loi_tui = w.copy(), loi
        lich_su.append((np.sum(du_doan(Xb, w) != y), loi_tui))
    return w, w_tui, lich_su

w_cuoi, w_tui, lich_su_tui = huan_luyen_tui(Xb_tr, y_tr, t_max=100)
print("w_cuoi =", w_cuoi, "| w_tui =", w_tui)
print("loi tren train: w_cuoi =", lich_su_tui[-1][0], "| w_tui =", lich_su_tu
i[-1][1], "/", len(y_tr))
```

```
w_cuoi = [-56.     12.73  -21.572] | w_tui = [-52.     18.135 -19.169]
loi tren train: w_cuoi = 69 | w_tui = 23 / 280
```

Hình 11 cho thấy lý do cần thuật toán túi. Số mẫu sai của **w** hiện tại **dao động rất mạnh** (có epoch chỉ sai khoảng 25 mẫu nhưng có epoch sai hơn 100 mẫu), nên nếu dừng ở một epoch bất kỳ ta có thể nhận về một bộ **w** rất tệ. Trong khi đó, số mẫu sai của $\mathbf{w}_{túi}$ **chỉ giảm, không bao giờ tăng**, và đạt 23 mẫu sai trên 280 mẫu huấn luyện.

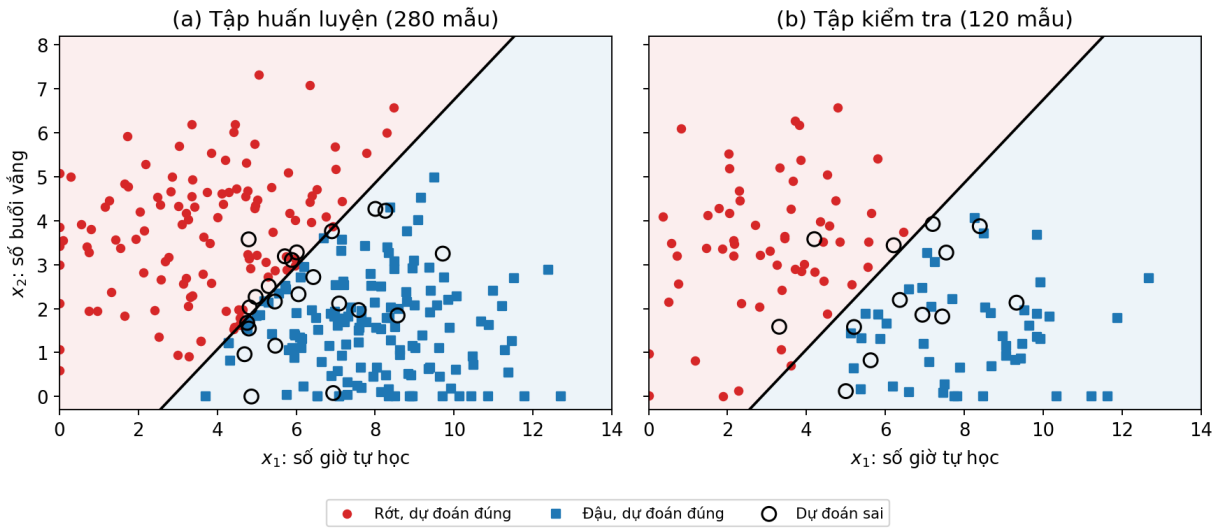


Hình 11. Số mẫu sai trên tập huấn luyện theo epoch. Đường đỏ (bộ ***w*** *hiện tại) dao động mạnh vì dữ liệu không phân tách tuyến tính được; đường xanh (bộ* $\mathbf{w}_{túi}$*) chỉ giảm dần.*

Đường biên tìm được

Ta dùng hàm *ve_duong_bien* (hoặc phiên bản vẽ riêng cho từng tập) để xem đường biên của $\mathbf{w}_{túi}$ trên cả hai tập. Các mẫu bị phân loại sai được khoanh tròn:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.6))
ve_duong_bien(ax1, X_tr, y_tr, w_tui, x1_max=14, x2_max=8)
ve_duong_bien(ax2, X_te, y_te, w_tui, x1_max=14, x2_max=8)
```



Hình 12. Đường biên của $\mathbf{w}_{túi}$ trên tập huấn luyện (a) và tập kiểm tra (b). Vùng xanh là vùng dự đoán đậu, vùng đỏ là vùng dự đoán rớt; các điểm khoanh tròn là mẫu bị dự đoán sai, tập trung ở khu vực hai lớp chồng lấn.

Các mẫu sai tập trung ở **vùng hai lớp chồng lấn nhau**, nơi không đường thẳng nào có thể tách đúng hết. Đây là giới hạn của dữ liệu, không phải lỗi của thuật toán.

Đo độ chính xác (accuracy)

Thước đo đơn giản nhất là **độ chính xác (accuracy)**: tỉ lệ mẫu được dự đoán đúng. Với n mẫu:

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left[\hat{y}_i = y_i\right]$$

Trong đó $\mathbb{1}[\cdot]$ bằng 1 nếu điều kiện đúng và bằng 0 nếu sai. Nhờ nhãn $\{-1, +1\}$, ta có thể viết gọn hơn: tích $y_i\hat{y}_i$ bằng $+1$ khi đúng và -1 khi sai (mục 1.1), nên $\frac{1+y_i\hat{y}_i}{2}$ bằng 1 khi đúng và 0 khi sai:

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \frac{1 + y_i\hat{y}_i}{2}$$

Trong mã, accuracy chỉ là *np.mean(y == y_hat)*. Ta tính cho ba bộ tham số trên cả hai tập:

```
acc = lambda a, b: np.mean(a == b)
for ten, ww in [("w (khong xao tron)", w_1), ("w_cuoi (xao tron)", w_cuoi), (
"w_tui", w_tui)]:
    print(f"{ten:20s} train: {acc(y_tr, du_doan(Xb_tr, ww)):.4f}  test: {acc(
y_te, du_doan(Xb_te, ww)):.4f}")
```

```
w (khong xao tron)   train: 0.8821  test: 0.8583
w_cuoi (xao tron)    train: 0.7536  test: 0.8250
w_tui                train: 0.9179  test: 0.8917
```

Bộ tham số	Accuracy trên tập huấn luyện	Accuracy trên tập kiểm tra
$\mathbf{w}$ của thuật toán thuần túy (không xáo trộn, dừng ở epoch thứ 100)	88,2%	85,8%
$\mathbf{w}$ ở epoch cuối (có xáo trộn)	75,4%	82,5%
$\mathbf{w}_{\text{túi}}$ (thuật toán túi)	91,8%	89,2%

Có ba nhận xét:

- **Thuật toán túi cho kết quả tốt nhất và ổn định nhất.** Bộ $\mathbf{w}$ ở epoch cuối bị dao động nên có thể rất tệ (chỉ 75,4% trên tập huấn luyện).
- Accuracy trên tập kiểm tra của $\mathbf{w}_{\text{túi}}$ (89,2%) thấp hơn một chút so với tập huấn luyện (91,8%). Khoảng chênh nhỏ này là bình thường: mô hình luôn "quen" dữ liệu huấn luyện hơn. Nếu khoảng chênh **rất lớn** thì mô hình đang **học thuộc** dữ liệu huấn luyện (overfit).
- Tập kiểm tra chỉ có 120 mẫu nên con số 89,2% cũng có sai số ngẫu nhiên. Đổi cách chia dữ liệu có thể làm nó thay đổi vài điểm phần trăm.

Ma trận nhầm lẫn (confusion matrix)

Accuracy chỉ cho biết **bao nhiêu** mẫu sai, không cho biết sai **kiểu nào**. Như ở mục 1.1, có hai loại sai khác nhau về bản chất. Ta quy ước **lớp dương là lớp "đậu"** ($y = +1$) và gọi tên bốn khả năng:

	Dự đoán $\hat{y} = +1$ (đậu)	Dự đoán $\hat{y} = -1$ (rớt)
Thực tế $y = +1$ (đậu)	**TP** (true positive): đoán đậu, đúng	**FN** (false negative): **bỏ sót**, đậu mà đoán rớt
Thực tế $y = -1$ (rớt)	**FP** (false positive): **báo nhầm**, rớt mà đoán đậu	**TN** (true negative): đoán rớt, đúng

Bốn số này được gom thành **ma trận nhầm lẫn C** kích thước 2×2, với **hàng là nhãn thực** và **cột là nhãn dự đoán**, thứ tự lớp $(-1, +1)$:

$$\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$$

Tổng bốn phần tử bằng số mẫu n, và accuracy được tính bằng tổng đường chéo chia tổng tất cả:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Hàm cài đặt và kết quả trên tập kiểm tra với $\mathbf{w}_{\text{túi}}$:

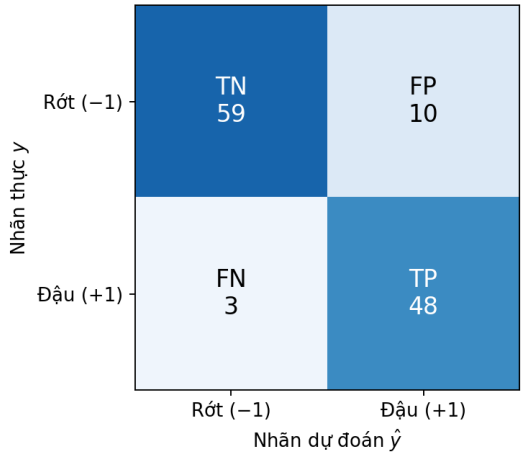
```
def ma_tran_nham_lan(y, y_hat):
    """Ma trận nhầm lẫn C = [[TN, FP], [FN, TP]] (hàng: nhãn thực, cột: nhãn dự đoán)."""
    tp = np.sum((y == 1) & (y_hat == 1))
    tn = np.sum((y == -1) & (y_hat == -1))
    fp = np.sum((y == -1) & (y_hat == 1))
```

```
    fn = np.sum((y == 1) & (y_hat == -1))
    return np.array([[tn, fp], [fn, tp]])

y_hat_te = du_doan(Xb_te, w_tui)
C = ma_tran_nham_lan(y_te, y_hat_te)
print(C)
```

```
[[59 10]
 [ 3 48]]
```

$$\mathbf{C} = \begin{bmatrix} 59 & 10 \\ 3 & 48 \end{bmatrix}$$



*Hình 13. Ma trận nhầm lẫn của $\boldsymbol{w}_{túi}$ trên tập kiểm tra (*120 *mẫu).*

Đọc kết quả: trong 120 sinh viên kiểm tra, mô hình đoán đúng 59 sinh viên rớt và 48 sinh viên đậu. Nó **báo nhầm** 10 sinh viên (rớt nhưng đoán đậu) và **bỏ sót** 3 sinh viên (đậu nhưng đoán rớt). Accuracy là $\frac{48+59}{120} = \frac{107}{120} \approx 89{,}2\%$, khớp với kết quả ở trên. Mô hình mắc lỗi **báo nhầm nhiều hơn bỏ sót**, nghĩa là nó có xu hướng "dễ dãi" khi dự đoán đậu.

Precision, recall và F1

Từ ma trận nhầm lẫn, ta tính các chỉ số chi tiết hơn cho **lớp dương**:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$= \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}$$

- **Precision (độ chính xác của dự đoán dương)** trả lời: *trong số sinh viên mô hình dự đoán là đậu, bao nhiêu phần trăm thật sự đậu?* Precision thấp nghĩa là mô hình hay **báo nhầm**.
- **Recall (độ phủ, hay độ nhạy)** trả lời: *trong số sinh viên thật sự đậu, mô hình phát hiện được bao nhiêu phần trăm?* Recall thấp nghĩa là mô hình hay **bỏ sót**.
- **F1** là trung bình điều hòa của hai chỉ số trên, chỉ cao khi cả hai cùng cao.

Với TP $= 48$, FP $= 10$, FN $= 3$:

$$\text{Precision} = \frac{48}{48+10} \approx 0{,}828, \qquad \text{Recall} = \frac{48}{48+3} \approx 0{,}941, \qquad \text{F1} = \frac{2 \cdot 48}{2 \cdot 48 + 10 + 3}$$
$$= \frac{96}{109} \approx 0{,}881$$

Mô hình phát hiện được 94,1% sinh viên đậu (recall cao), nhưng chỉ 82,8% số dự đoán "đậu" là đúng (precision thấp hơn).

Khi nào cần chỉ số nào? Tầm quan trọng của hai loại sai phụ thuộc vào bài toán:

- Tầm soát bệnh: **bỏ sót** bệnh nhân (FN) nguy hiểm hơn nhiều so với báo nhầm, nên ta ưu tiên **recall** cao.
- Lọc thư rác: **báo nhầm** thư quan trọng là thư rác (FP) gây thiệt hại hơn, nên ta ưu tiên **precision** cao.

Cạm bẫy của accuracy với dữ liệu lệch lớp. Giả sử trong 100 mẫu có 95 mẫu âm và 5 mẫu dương, và một mô hình "lười" luôn dự đoán -1. Khi đó TN $= 95$, FN $= 5$, TP $= $ FP $= 0$ nên

$$\text{Accuracy} = \frac{0+95}{100} = 95\%, \qquad \text{Recall} = \frac{0}{0+5} = 0$$

Accuracy 95% trông rất tốt, nhưng mô hình không phát hiện được **mẫu dương nào**. Vì vậy, với dữ liệu lệch lớp, ta luôn cần xem ma trận nhầm lẫn, precision và recall chứ không chỉ accuracy.

5.3. Huấn luyện và so sánh với Perceptron trong scikit-learn

Trong thực tế, ta ít khi tự cài đặt mà dùng thư viện **scikit-learn**. Việc cài đặt từ đầu ở mục 5.1 vẫn có giá trị vì nó giúp hiểu rõ thư viện làm gì bên trong. Lớp *Perceptron* của scikit-learn dùng như sau:

```
from sklearn.linear_model import Perceptron

clf = Perceptron(eta0=1.0, max_iter=20, tol=None, shuffle=False)
clf.fit(X_sv, y_sv)
print("intercept_ =", clf.intercept_, "| coef_ =", clf.coef_)
```

```
intercept_ = [-1.] | coef_ = [[ 4. -8.]]
```

Các tham số cần chú ý:

- *eta0=1.0*: tốc độ học η.
- *max_iter*: số epoch tối đa $T_{\max}$.
- *tol=None*: **tắt** cơ chế dừng sớm của scikit-learn, để thuật toán chạy đủ *max_iter* epoch như mã của ta.
- *shuffle=False*: **không** xáo trộn thứ tự mẫu, để có thể so với mã của ta.

Lưu ý scikit-learn **không** cần ta thêm cột 1: ta đưa thẳng *X_sv* (ma trận $n \times d$), và hệ số chệch w_0 được lưu **riêng** trong *intercept_*, còn các trọng số $w_1, \ldots, w_d$ nằm trong *coef_* (mảng có

dạng $1 \times d$). Ghép lại, $\mathbf{w} = [-1 \quad 4 \quad -8]^T$, **đúng bằng kết quả của mã tự cài đặt** và của ví dụ tính tay.

Kiểm chứng trên dữ liệu lớn

Ta chạy lại với dữ liệu huấn luyện lớn, cùng cấu hình, và so với mã tự cài đặt (*w_1* ở mục 5.2):

```
clf = Perceptron(eta0=1.0, max_iter=100, tol=None, shuffle=False)
clf.fit(X_tr, y_tr)
print("sklearn: intercept_ =", clf.intercept_, "coef_ =", clf.coef_)
print("tu cai dat: w =", w_1)
```

```
sklearn: intercept_ = [-65.] coef_ = [[ 27.118 -27.472]]
tu cai dat: w = [-65.     27.118 -27.472]
```

Hai kết quả **trùng nhau**, nghĩa là thuật toán Perceptron trong scikit-learn thực chất chính là thuật toán mà ta đã viết: cập nhật khi $m_i \leq 0$, bước cập nhật $\eta\, y_i\, \mathbf{x}_i^T$, khởi tạo $\mathbf{w} = \mathbf{0}$, duyệt theo epoch.

Những khác biệt cần biết

Điểm	Mã tự cài đặt	scikit-learn
Hệ số chệch	Là phần tử đầu w_0 của w (cần *them_bias*)	Lưu riêng trong *intercept_*, còn *coef_* chứa $w_1, \ldots, w_d$
Mẫu có $s = 0$	Dự đoán $+1$ (quy ước $\text{sign}(0) = +1$)	Dự đoán lớp **thứ nhất** (ở bài này là -1) vì chỉ gán lớp dương khi $s > 0$
Thứ tự duyệt mẫu	Cố định (hoặc xáo trộn nếu ta cài đặt)	Mặc định **xáo trộn** (*shuffle=True*)
Điều kiện dừng	$c = 0$ hoặc $T_{\max}$	Mặc định dừng sớm khi cải thiện nhỏ hơn *tol* (*tol=1e-3*)
Nhãn	$\{-1, +1\}$	Nhận hai nhãn bất kỳ, lưu trong *classes_*

Khác biệt ở mẫu có $s = 0$ rất hiếm gặp trên dữ liệu thật, nhưng ta đã kiểm tra được trên mẫu $\mathbf{x} = [2 \quad 0{,}875]$ (điểm số đúng bằng 0): mã của ta dự đoán $+1$, còn scikit-learn dự đoán -1.

So sánh độ chính xác

Ta so ba mô hình trên cùng tập huấn luyện và tập kiểm tra: Perceptron tự cài đặt (thuật toán túi), *Perceptron* của scikit-learn với cấu hình mặc định, và **hồi quy tuyến tính rồi đặt ngưỡng** 0 (ý tưởng ở mục 1.2):

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score

clf_mac_dinh = Perceptron(random_state=0).fit(X_tr, y_tr)
hq = LinearRegression().fit(X_tr, y_tr)
du_doan_hq = lambda X_: np.where(hq.predict(X_) >= 0, 1, -1)
```

```
rows = [("Perceptron tu cai dat (w_tui)", du_doan(Xb_tr, w_tui), du_doan(Xb_te, w_tui)),
        ("sklearn Perceptron (mac dinh)", clf_mac_dinh.predict(X_tr), clf_mac_dinh.predict(X_te)),
        ("Hoi quy tuyen tinh + nguong 0", du_doan_hq(X_tr), du_doan_hq(X_te))
]
for ten, a, b in rows:
    print(f"{ten:32s} train {accuracy_score(y_tr, a):.4f}  test {accuracy_score(y_te, b):.4f}")
```

```
Perceptron tu cai dat (w_tui)    train 0.9179  test 0.8917
sklearn Perceptron (mac dinh)    train 0.9071  test 0.8917
Hoi quy tuyen tinh + nguong 0    train 0.9071  test 0.8917
```

Mô hình	Accuracy huấn luyện	Accuracy kiểm tra
Perceptron tự cài đặt (thuật toán túi)	91,8%	89,2%
Perceptron của scikit-learn (mặc định)	90,7%	89,2%
Hồi quy tuyến tính + ngưỡng 0	90,7%	89,2%

Ba mô hình cho kết quả **gần như bằng nhau** trên tập kiểm tra. Điều này không mâu thuẫn với mục 1.2. Ở đây dữ liệu khá cân bằng và không có mẫu ngoại lai cực đoan, nên hồi quy tuyến tính rồi đặt ngưỡng hoạt động tạm ổn. Điểm yếu mà mục 1.2 chỉ ra (một mẫu "rất đúng" hay một mẫu ngoại lai có thể làm lệch ngưỡng) chỉ lộ ra trong những tình huống như vậy, và nó là lý do ta cần một mô hình và hàm mất mát được thiết kế riêng cho phân loại. Quan trọng hơn, **không mô hình nào ở đây vượt qua được giới hạn của dữ liệu chồng lấn**: cả ba đều dừng quanh 89% trên tập kiểm tra, vì đường biên phẳng không thể tách hoàn toàn hai lớp chồng lấn.

Đánh giá bằng các hàm có sẵn

Các chỉ số ở mục 5.2 đều có hàm dựng sẵn trong *`sklearn.metrics`*. Ta kiểm tra rằng kết quả trùng với phép tính tay:

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_te, y_hat_te, labels=[-1, 1]))
print(classification_report(y_te, y_hat_te, labels=[-1, 1],
                            target_names=["rot", "dau"], digits=3))
```

```
[[59 10]
 [ 3 48]]
              precision    recall  f1-score   support

         rot      0.952     0.855     0.901        69
         dau      0.828     0.941     0.881        51

    accuracy                          0.892       120
   macro avg      0.890     0.898     0.891       120
weighted avg      0.899     0.892     0.892       120
```

Dòng "dau" cho precision 0,828, recall 0,941, F1 0,881, **đúng bằng** các giá trị ta tính tay. Dòng "rot" là các chỉ số tính cho lớp âm (xem lớp rớt là lớp "dương"): precision $\frac{59}{59+3} \approx 0{,}952$ và recall $\frac{59}{59+10} \approx 0{,}855$.

> **Lời khuyên thực hành.** Khi dùng Perceptron (hay các mô hình tuyến tính nói chung) trên dữ liệu thật, nên **chuẩn hóa đặc trưng** (ví dụ bằng `StandardScaler`) để các đặc trưng có thang đo tương đương, và luôn **báo cáo kết quả trên tập kiểm tra**, không phải trên tập huấn luyện.

Tóm tắt mục 5

- Perceptron được cài đặt bằng khoảng 20 dòng NumPy. Mỗi công thức có một dòng mã tương ứng: $\mathbf{Xw}$ là `Xb @ w`, margin m_i là `y[i] * (Xb[i] @ w)`, cập nhật là `w = w + eta * y[i] * Xb[i]`. Kết quả khớp hoàn toàn với ví dụ tính tay ở mục 3.5.
- Cần **chia dữ liệu** thành tập huấn luyện (để học) và tập kiểm tra (để đánh giá, chỉ dùng cuối cùng) nhằm đo khả năng dự đoán dữ liệu mới.
- Dữ liệu thật thường chồng lấn nên Perceptron **không hội tụ** và $\mathbf{w}$ dao động. **Thuật toán túi** giữ lại bộ $\mathbf{w}$ tốt nhất và cho kết quả ổn định hơn.
- **Accuracy** là tỉ lệ dự đoán đúng, nhưng có thể gây hiểu lầm khi dữ liệu lệch lớp. **Ma trận nhầm lẫn** $\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$ cho biết kiểu sai, từ đó tính **precision** $= \frac{\text{TP}}{\text{TP+FP}}$, **recall** $= \frac{\text{TP}}{\text{TP+FN}}$ và **F1**.
- `Perceptron` của scikit-learn thực hiện cùng thuật toán. Khi cấu hình `shuffle=False`, `tol=None`, kết quả **trùng khớp** với mã tự cài đặt. Khác biệt chính nằm ở cách lưu hệ số chệch, thứ tự duyệt mẫu mặc định và điều kiện dừng.

Câu hỏi kiểm tra nhanh

1. Vì sao ta không nên đánh giá mô hình trên chính tập dữ liệu đã dùng để huấn luyện?
2. Cho ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} 40 & 5 \\ 10 & 45 \end{bmatrix}$ (hàng: nhãn thực $-1, +1$; cột: nhãn dự đoán $-1, +1$). Hãy tính accuracy, precision, recall và F1 cho lớp dương.
3. Một mô hình tầm soát bệnh có accuracy 98% trên bộ dữ liệu có 2% người bệnh. Vì sao con số này chưa đủ để kết luận mô hình tốt? Ta cần xem chỉ số nào?
4. Trong hàm `huan_luyen`, nếu ta đổi điều kiện `if m <= 0:` thành `if m < 0:` và khởi tạo `w = np.zeros(d1)`, điều gì xảy ra khi chạy trên dữ liệu bất kỳ? (gợi ý: mục 3.2 và câu 3 của mục 3)
5. Để đưa ra dự đoán cho một sinh viên mới học 6 giờ, vắng 2 buổi bằng bộ $\mathbf{w} = [-1 \quad 4 \quad -8]^T$, ta viết đoạn mã Python nào? Kết quả là gì?
6. Vì sao khi dùng scikit-learn, ta phải truyền `X` **không** có cột 1, còn với mã tự cài đặt ta phải gọi `them_bias` trước?

Gợi ý đáp án

1. Vì mô hình đã "nhìn thấy" dữ liệu huấn luyện, nên độ chính xác trên đó thường cao hơn so với trên dữ liệu mới và không phản ánh khả năng tổng quát hóa. Tập kiểm tra được giấu đi trong lúc huấn luyện nên cho ước lượng trung thực hơn.

2. TN $= 40$, FP $= 5$, FN $= 10$, TP $= 45$, tổng $= 100$. Accuracy $= \frac{45+40}{100} = 85\%$. Precision $= \frac{45}{45+5} = 0{,}90$. Recall $= \frac{45}{45+10} \approx 0{,}818$. F1 $= \frac{2 \cdot 45}{2 \cdot 45+5+10} = \frac{90}{105} \approx 0{,}857$.
3. Một mô hình luôn dự đoán "không bệnh" đã có accuracy 98% nhưng bỏ sót toàn bộ bệnh nhân (recall $= 0$). Ta cần xem **recall** (và precision, ma trận nhầm lẫn) của lớp bệnh, vì bỏ sót bệnh nhân (FN) rất nguy hiểm.
4. Với $\mathbf{w} = \mathbf{0}$, mọi mẫu có $m_i = 0$ nên điều kiện $m < 0$ không bao giờ đúng. Thuật toán không cập nhật lần nào, epoch đầu tiên có $c = 0$ và dừng ngay tại $\mathbf{w} = \mathbf{0}$, một nghiệm vô nghĩa.
5. Ví dụ: `x = np.array([1, 6, 2]); w = np.array([-1, 4, -8]); s = x @ w; y_hat = 1 if s >= 0 else -1`. Điểm số $s = -1 + 24 - 16 = 7 \geq 0$ nên $\hat{y} = +1$ (dự đoán đậu).
6. scikit-learn tự quản lý hệ số chệch bằng tham số `fit_intercept=True` và lưu w_0 riêng trong `intercept_`, nên ta chỉ truyền các đặc trưng gốc. Mã tự cài đặt gộp w_0 vào `w`, nên cần thêm cột 1 vào `X` để **xw** tự động cộng w_0.